Protocol, part 1 of 2

Protocol, part 1 of 2

This part contains 24 notes, from `protocol/chat-pm-rc.md` through `protocol/packet-layouts.md`.

Notes under `protocol/`, in path order.

`protocol/chat-pm-rc.md`

Chat, private messages, and staff traffic

Date: 2026-10-21

Sources: GServer-v2 `TPlayer.cpp`, `TPlayerRC.cpp` (GPLv3, summarized); 2.05 tables. Tags as elsewhere.

Chat bubble

How the line is drawn and how long it stays, per build: `client-internals/chat-text.md`.

Local chat is not a message id. It is the `PLPROP_CURCHAT` (12) string inside `PLI_PLAYERPROPS`. The server forwards it to the level as `PLO_OTHERPLPROPS`. Server commands typed in chat (`setnick`, `/sethead`, …) are parsed from the same prop. **[srv]** The one exception is toall: when the chat-bar text (lower-cased and trimmed) starts with `toall:`, 2.05 sends the rest as `PLI_TOALL` (6) instead of as CURCHAT. **[2.05]**

Toall

Dir	Id	Layout
C→S	6 TOALL	`GSTRING message` (2.05: text after `toall:`, trimmed, max 200 bytes)
S→C	13 TOALL	`s player id`, `str message` (2.05 handles it)

Private messages

Dir	Id	Layout	2.05
C→S	28 PRIVATEMESSAGE	`s count`, `count × s player id`, rest = message (gtokenized)	✓
S→C	37 PRIVATEMESSAGE	`s from id`, rest = `"type"`,`"message"` as a quoted, comma-separated list. The type is a label such as `"Server Message:"`, `"Guild message:"`, or empty	✓

Mass PMs and guild PMs use the same packet with different type labels. Players in jail can PM only RCs. **[srv]**

Admin and staff

Dir	Id	Name	Notes	2.05
S→C	35	RC_ADMINMESSAGE	rest = `Admin <account>: \xa7<text>`	✓
S→C	47	STAFFGUILDS	`"guild",` repeated (quoted, comma-terminated), sent at login	✓

Dir	Id	Name	Notes	2.05
S→C	74	RC_CHAT	RC console line	·
C→S	51–98	RC_*	server options, folder config, player props get/set, accounts, bans, rights, comments, file browser	· (RC client only)
C→S	103–119	NC_*	NPC-control: NPC get/set/warp, classes, weapons	·

The staff tools run their own logins: RC `GSERV023–025`, NC `NCL11012`, RC2 type 6 on gen 5. NC packet layouts: `protocol/npc-control.md`. RC details: `tools/remotecontrol.md`.

Profiles

Dir	Id	Layout	2.05
C→S	80 PROFILEGET	rest = account	✓
C→S	81 PROFILESET	10 fields, each `c len` + text (see below) [2.05]	✓
S→C	75 PROFILE	the same field list, relayed from the listserver [2.05]	✓

Profile field list (2.05) [2.05]

Both PROFILESET and PROFILE are a run of length-prefixed strings, `c(len) + text` each, read until the message ends. The fields come from the profile window, in this order:

#	Field	Notes
0	account	the profile's owner
1	real name	
2	age	
3	sex	`-` (unknown), `male`, or `female`
4	country	
5	ICQ number	
6	e-mail	
7	homepage	
8	favourite Graal hangout	
9	favourite quote	the quote box's lines joined with spaces, trimmed, cut to 220 characters

Each text field is trimmed before sending. 2.05 sends PROFILESET from the profile window's Send button. Close on your own profile only keeps the edits locally. The Send button also works in another player's profile window (while that player is online), and then field 0 is **that player's** account, so a server must not take field 0 as proof of who is editing.

On PROFILE, 2.05 needs at least 10 fields (fewer and the packet is ignored). It drops field 0 and saves fields 1–9 to a local per-account profile file. The in-game stats in the same window (online time, kills, deaths, rating, alignment, max power, gralats, sword power, spin, level) are not in this packet; they most likely come from the player's props [inf].

Open

- Whether 2.05 shows `PLO_STAFFGUILDS` anywhere.

`protocol/client-versions.md`

Client versions side by side: 2.05, 2.17, 2.22, 2.31

Date: 2026-10-21

Sources: the four client executables in the Graal Reborn 2.2.2.0 folder (`Graal.exe` = 2.05 rev 2, `Graal_2171.exe`, `Graal_222.exe`, `Graal_231.exe`), read statically. Server-side names are from gs2lib. Tags: **[bin]** read from the client binaries · **[srv]** server source only · **[doc]** `newfeatures*.txt` · **[inf]** inferred.

All four clients are Delphi builds with the same network classes: one base socket class, a game-server subclass, and a listserver subclass. They also share the same cipher LCG, `it = it * 0x08088405 + key` with seed `0x04A80B38`. It uses the Borland `Random` multiplier, but adds the key where Borland adds 1. **[bin]**

Game-server login

	2.05 rev 2	**2.17 (2.1.7.1)**	**2.22**	**2.31**
Login type (first GCHAR)	0	0	**5**	**5**
Key	GCHAR `rand & 0x7F`	same	same	same
Token	`GNW03081`	`GNW22122`	`GNW03014`	`GNW28015`
Then	`str account`, `str password`	same	same	same
Identity string after password	no	no	no	no
Cipher (client → server)	gen 3 insertion	**gen 3 insertion**	gen 5 partial XOR	gen 5 partial XOR
Frame (client → server)	`u16be len` + zlib	same	`u16be len` + compression byte + body	same
Sent right after login	`PLI_LANGUAGE` (37)	37, then `PLI_TAMPERCHECK` (39)	37, 39	37, 39

[bin]

- The token is assembled one character at a time in every build, so none of them has it as a string literal. **[bin]**
- **2.17 still uses the 2.05 cipher.** Its game-server send path is the same gen-3 code: a junk byte at `(it & 0xFFFF) mod len` from the second message on, then zlib over the buffer. The out-buffer limits are the same (flush at 0x1000, drop at 0x40000 queued). The gs2lib numbering puts type 4 (gen 4) at 2.19–2.21, which fits 2.17 still being type 0. **[bin][srv]**
- 2.22 and 2.31 send the login message in a fixed "zlib, no cipher, no compression byte" mode, flush it, and only then switch the socket to the dynamic gen-5 mode. So the login frame has no compression byte, and every later frame has one. **[bin]**
- The 2.22+ login line ends at the password. The guess in `login-and-walk.md` that 2.22+ appends an identity string does not hold for 2.22 or 2.31. **[bin]**
- `PLI_TAMPERCHECK` (39) carries **10 raw bytes**, not two GINT5 words: the bytes include values below 0x20, which a GINT5 byte can never be, and none is 0x0a, so the payload survives line framing unescaped. The 10 bytes are a constant per client build, identical across accounts and login keys (2.17 `2a7bad195e231243d82c`, 2.22 `250e4d0f962b03ede631`, 2.31 `291655b7602e87c9eb82`; see `version-profiles/tampercheck-values.tsv`). It is new in 2.17 and absent from 2.05. The bytes are two chained CRC-32 words over the client's own memory image, each written in a 5-byte non-GINT5 form with unmasked middle bytes, so they only change if the exe changes (`client-internals/decomp-`

`findings-2026-10-24-b/01-tamper-check-id39.md`, words in `version-profiles/tampercheck-words.tsv`). 2.31 re-sends it every 3.2 s because a 50 ms timer counts modulo 64. **[bin][cap]**

- From 2.17 on, the account name is kept obfuscated in memory and decoded only to build the login line. 2.31 does the same for the password. The obfuscation is the same byte map used for hidden strings, `dec(b) = (~b − 10) & 0xFF`. **[bin]** The 2.18 note "new password encryption" refers to the saved-password store, not to this. **[doc][inf]**

Gen-5 send on the client (2.22, 2.31, RemoteControl) [bin]

The server-side description in `codec-and-framing.md` is the receive half. On the client, each flush of the out buffer (`n` bytes) picks its own compression:

Buffer size	Type byte	Body
`n < 0x30`	2, or 4 if zlib is actually shorter	raw (or zlib)
`0x30 ≤ n < 0x1000`	4	zlib
`n ≥ 0x1000`	6	bz2

Then:

1. Partial XOR over the (compressed) body: `min(ceil(n/4), 12)` dwords for type 2 and `min(ceil(n/4), 4)` dwords for types 4 and 6, that is one LCG step per 4-byte group present, up to 12 groups uncompressed or 4 groups compressed. Each dword advances the LCG once and XORs with it, little-endian. A short last group still steps the LCG and XORs byte by byte with the iterator's low bytes. *(An earlier version of this note had `floor(n/4)` for types 4 and 6; that was wrong, see `codec-and-framing.md` for the evidence and which side it covers. The client's own send helper has not been re-read for this correction, and no real-client send of a short compressed frame has been captured.)*
2. The frame is `u16be (len + 1)`, then the type byte, then the body.

The receive side mirrors this with its own iterator. Both iterators start at `0x04A80B38` and use the same key. Type codes 1/2 mean raw, 3/4 zlib, and 5/6 bz2, and only the even codes are XORed. The odd code 3 is the fixed "zlib, no cipher" mode used for the login frame and for the whole NC session (see `tools/remotecontrol.md`). The client drops the connection once more than 0x40000 bytes are queued (RemoteControl: 0x800000).

Server → client ids handled

Read from each client's main receive switch. ✓ = has a case. **[bin]**

Ids	2.05	2.17	2.22	2.31	Notes
0–25, 27–43, 45–48	✓	✓	✓	✓	the 2.05 core set
26 `NPCACTION`	·	·	·	·	never handled
44 `HASNPCSERVER`	✓	✓	✓	·	dropped in 2.31 ("Classic switched to NPC-Server", v2.3) **[doc]**
49 `PLAYERWARP2`	·	✓	✓	✓	gmap warp
68 / 69 `LARGEFILESTART` / `END`	·	✓	✓	✓	"files bigger than 64k can be downloaded" (v2.14) **[doc]**
75 `PROFILE`	✓	✓	✓	✓	
84 `LARGEFILESIZE`	·	·	·	✓	`downloadsize` "correctly giving the size" (v2.3) **[doc]**

Ids	2.05	2.17	2.22	2.31	Notes
100 `RAWDATA`	✓	✓	✓	✓	101/102 arrive behind it
150, 151, 153–156, 165, 166, 168	✓	✓	✓	✓	
169 `UPGRADEURL` … 185 `BOARDHEIGHTS`	·	✓	✓	✓	ghost mode (170, 173, 174), bigmap/ minimap from the gserver (171, 172), Kingdoms and New World modes (176, 177), `serverwarp` (178), RPG window (179), status list (180), process list and screenshot (182–184), level heights (185)
186 `BOARDMODIFY2`	·	·	✓	✓	
189 `MOVE2`	·	·	·	✓	"full precision" positions and NPC moves (v2.3) **[doc]**

Counts: 2.05 handles 59 ids, 2.17 handles 79, 2.22 handles 80, and 2.31 handles 81 (per-build lists for 2.00 to 2.31: `version-profiles/plo-handled-ids-by-build.tsv`; payload layouts: `protocol/packet-layouts.md` and `version-profiles/plo-layouts-217-231.tsv`). All of them ignore unknown ids without error. The names come from gs2lib. That a client has a case for an id shows it handles it. Its payload is not traced here. **[bin][srv]**

Listserver login

	2.05	2.17	2.22	2.31
Host and port	built in, DES-hidden `<list-host-classic>`, `14921`	`arrays2.dat`	`arrays2.dat`	`arrays2.dat`
First message	id 0 `newmain`	id 0 `newmain`	id 7, key, `GNW30123newmain`	id 7, key, `GNW30123newmain`
Then	nothing: 2.05 sends only id 0 and waits for the list	id 1 `str account, str password`	id 1, now gen 5	id 1, now gen 5
Frame	`u16be` + zlib, no cipher	same	gen 5 after the first message	gen 5 after the first message

[bin]

- **`GNW30123` is the 2.22/2.31 listserver token.** It travels inside listserver id 7 (`V2ENCRYPTKEYCL`), straight after the key, and is immediately followed by the list name `newmain`. So it is a listserver protocol token, not a game build, which is why no game-version table has it. **[bin][srv]**
- From 2.17 the listserver address comes from `arrays2.dat`, a CRLF text file checked against the 128-byte signature in `arrays2.sig`. The verification key is a long hex string hidden in the exe with the same `~b −10` map. The signature algorithm is not verified here (RSA-1024 is a plausible reading of the key and signature sizes) **[inf]**. Lines: 0 = primary host, 1 = port, 2 = site domain, 3 = account-registration URL, 4 = translation URL, 5 and 6 = fallback hosts. The client tries up to three hosts in turn. **[bin]** The Graal Reborn copy reads `<list-host-reborn>`, port `14922`, with fallbacks `<list-host-graalcc>` and `<list-host-graalbiz>`. `graal.sig` is checked the same way against the executable itself, which is the "Graal.exe corrupted" check in `readme.txt`. **[bin][doc]**

- 2.05 has no `arrays2.dat` or `.sig` handling at all. **[bin]**

Open

- Client → server id sets per version. The send helpers are virtual calls with the id in a register, so a full table needs per-version helper identification. 2.05's set is in `packet-ids.md`.
- ~~What the two `PLI_TAMPERCHECK` words are.~~ Answered 2026-10-24 (see the tamper bullet above). Still open: whether any server enforces the value.
- Client → server ids 36, 42 and 45 in 2.17.1, 2.22 and 2.31 are still unmatched to call sites (`client-internals/decomp-findings-2026-10-24-b/05-open-items.md`).
- 2.19–2.21 (type 4, gen 4) are not in this folder, so the gen-4 client side is unverified.

`protocol/codec-and-framing.md`

Codec, framing, and encryption generations

Date: 2026-10-21

Sources: gs2lib `CEncryption`, `CFileQueue`, `CString` (GPLv3, summarized); GServer-v2 `TPlayer` receive path; client 2.05 rev 2 (`GNW03081`) send path; 2026-10-04 captures.

Confidence tags: **[2.05]** confirmed in the 2.05 binary · **[srv]** server source only · **[cap]** seen in captures · **[inf]** inferred.

Graal integer types

All "G" types offset each 7-bit group by 0x20 so the stream stays printable and never contains `\n`.

Type	Bytes	Encode	Range	Tag
GCHAR	1	`v + 0x20`	0–223	[srv][2.05]
GSHORT	2	`(v >> 7) + 0x20`, `(v & 0x7F) + 0x20`	0–28767	[srv]
GINT (GINT3)	3	7-bit groups, high first, each `+ 0x20`	0–3682399	[srv]
GINT4	4	same, four groups	—	[srv]
GINT5	5	first group 4 bits (`>> 28`), then four 7-bit groups	32-bit	[srv]
GSTRING	1 + n	GCHAR length then bytes	≤ 223 bytes	[srv][2.05]

The server clamps rather than wraps on overflow (GSHORT high byte ≤ 223). Signed values are sent as their unsigned bit pattern.

Messages

One message = `GCHAR(id)` + payload + `\n`. Several messages are concatenated into one frame. **[srv][2.05]**

The 2.05 sender refuses a payload that already contains `\n`. Binary payloads (board, files) use the raw-data escape below. **[2.05]**

Raw-data escape

`PLO_RAWDATA` (100, wire `0x84`) carries a GINT3 length. The next *length* bytes are read as one message, newline or not. The server uses it in front of `PLO_BOARDPACKET` (101) and `PLO_FILE` (102). **[srv]** 2.05 handles 100 in its main dispatch table but not 101 or 102. That fits with 101/102 only ever arriving behind 100. **[2.05][inf]**

Frame

```
u16be length | body
```

- Client to server in 2.05: the body is zlib-deflated. Messages collect in an out buffer, which is compressed and flushed at 0x1000 bytes or on an explicit flush. The 2.05 client links zlib 1.1.2 (Delphi ZLib unit). If more than 256 KB is queued unsent, the client drops the connection. **[2.05]**
- Server to client (gen 2/3): zlib body, no extra byte. **[srv][cap]**
- Gen 4: bz2 body. Gen 5: one compression byte after the length (`0x02` none, `0x04` zlib, `0x06` bz2), then the body. **[srv]**

Encryption generations (gs2lib numbering)

Gen	Enum	Used by	Scheme	Tag
1	`ENCRYPT_GEN_1`	web client	none, uncompressed	[srv]
2	`ENCRYPT_GEN_2`	client type 0 with a known token in the first 8 bytes	zlib, no cipher	[srv]
3	`ENCRYPT_GEN_3`	**2.05**, RC, NC, NPC-server, any type-0 client whose first 8 bytes are not a known token	zlib frame; one junk byte inserted into each message	[srv][2.05]
4	`ENCRYPT_GEN_4`	client type 4 (2.19–2.21, 3.0–3.01)	bz2, then XOR of the first *limit* dwords	[srv]
5	`ENCRYPT_GEN_5`	client type 5 (2.22+), RC2 (type 6)	compression byte, then the same partial XOR	[srv]
6	`ENCRYPT_GEN_6`	v6	unknown, not implemented	[srv]

Client-side check (2026-10-21) **[bin]**: 2.17 also sends type 0 with gen-3 insertion, like 2.05. 2.22 and 2.31 send type 5 and use gen 5. The Delphi RemoteControl logs in as type 6 (gen 5), and its NC socket as type 3, which stays zlib with no cipher. The client-side gen-5 send rules (when each compression byte is chosen, XOR word counts) are in `client-versions.md`.

Gen 3 (2.05)

State: `it` (u32), seeded `0x04A80B38`. Key `k` = 7-bit random value, sent once in the login line. **[2.05]**

Per outgoing message, starting with the **second** message of the session. Only the login line itself is sent clean. The messages that follow it in the same first frame (such as `PLI_LANGUAGE`) already carry the byte. **[2.05][cap]**

1. `it = it * 0x08088405 + k` (mod 2^32).
2. `pos = (it & 0xFFFF) mod len + 1` (1-based; `len` includes the trailing `\n`). `pos` can be 1, which puts the junk byte **before the id byte**, so a decoder must not assume the id is at offset 0 until the byte is removed. **[2.05][cap]**
3. Insert one byte `((it >> 16) & 0x7F) + 0x20` before the 1-based position `pos`.

`pos` can be 1, which puts the junk byte **before the id byte**. A decoder that skips the first byte as "the id" before looking for the junk byte gets that case wrong. Because the inserted byte is never `\n` and `pos` is at most `len`, the byte never lands after the terminating `\n`. **[2.05][cap]**

The server mirrors this: same LCG step, deletes the byte at 0-based `(it & 0xFFFF) mod len`. **[srv]** Here `len` is the line the server split off, which drops the `\n` and keeps the junk byte. That is the same length the client used, so the server's 0-based index is exactly where the client put the byte. GServer-v2 therefore decodes 2.05 correctly, with no tolerance tricks. **[srv][2.05]**

Wire check (lab, three 2.05 sessions): every client→server message after the login line had the expected byte `((it >> 16) & 0x7F) + 0x20` at the predicted index. Several messages had it at index 0, before the id. There were no mismatches. **[cap]** The server's own gen-3 encoder always inserts `)` instead of an iterator-derived byte. That is harmless because the receiver throws the byte away. **[srv]** Gen 3 runs per message *after* decompression, not over the whole frame. **[srv]**

Gen 3 is client→server only. Server→client traffic for a gen-3 client is plain zlib. **[srv][cap]** The 14842/14912 captures stay clear zlib, which matches this.

Gen 4/5 partial XOR

Same LCG and key. For each 4-byte group present in the body, advance `it` once and XOR the group with the little-endian bytes of `it`. A short last group counts as a group: it advances `it` and is XORed byte by byte with the low bytes. Stop after *limit* groups: 12 groups for uncompressed bodies (`0x02`), 4 for zlib or bz2 (`0x04`/`0x06`). The group count is therefore `min(ceil(len/4), limit)`, with no special case for compressed bodies. On send the body is compressed first and the XOR is applied to the compressed bytes. On receive the XOR is undone first, then the body is decompressed. **[srv][2.22]**

2.22 client side **[2.22][cap]**: - **Login frame:** `u16be len` + zlib, with no compression byte and no XOR. The key travels inside it, so it has to be readable. - **Compression choice for later frames:** - Bodies of 0x1000 bytes or more use bz2 (`0x06`). - Bodies from 0x30 bytes up use zlib (`0x04`). - Bodies under 0x30 bytes are zlib-compressed only when that makes them smaller; otherwise they go uncompressed (`0x02`). - **Lengths:** the `u16be` length counts the compression byte. - **Group counting:** one iterator step per 4-byte group present, capped at 12 groups for `0x02` and at 4 groups for `0x04`/`0x06`. A partial last group still steps the iterator and is XORed byte by byte. The count is `min(ceil(len/4), limit)` for every type. - **Correction (2026-10-24):** an earlier version of this note gave `floor(len/4)` capped at 4 for `0x04`/`0x06`. That is wrong. It skips one iterator step for any compressed body whose length is not a multiple of 4 (and all of them under 4 bytes), so every later frame in that direction decodes as noise. - **Evidence:** the gs2lib encryption routine **[srv]**; 27 of 27 server → client frames from four public 2.22 gservers plus both listserver frames **[cap]**, which includes zlib and bz2 bodies with a partial last group; and the Seihai lab sessions (610 frames, both directions, against GServer-v2 and the lab listserver) **[cap]**. These cover the **receive side of the client** (server → client frames) and the server's receive of Seihai's frames. **Client send side, confirmed 2026-10-24 [cap]:** a real 2.22 client under Wine, over several lab sessions, sent 580 uncompressed (`0x02`) and 17 zlib (`0x04`) frames. The 17 compressed bodies were 37, 37, 49, 49, 50, 50, 50, 51, 51, 51, 53, 53, 282, 286, 307, 313 and 315 bytes long; body length mod 4 was 1 (7 frames), 2 (5) and 3 (5), never 0. All 17 decode and decompress under the rule above (one step per group present, partial last group steps, cap 4 for zlib), and the old `floor(len/4)` rule fails on them. No `0x06` (bz2) frame was ever sent by the client in these sessions; the client bz2 send path (needs a send over 0x1000 bytes) is still unseen. - **Not seen yet:** a client-originated bz2 frame (body of 0x1000 bytes or more). - **Wire check:** a lab 2.22 session decoded completely in both directions with this rule. This is the cipher that makes the 14801 sessions stop inflating after the first frame (2.22/2.31 clients). **[cap][inf]**

Corrections to earlier notes

- "Some post-login frames are encrypted (Borland LCG)" in CONSOLIDATED.md is gen 4/5. It does not apply to 2.05. 2.05 uses gen 3 (insertion only).
- "First line of a session inserts one extra LCG byte" (crosscheck note): the first line carries the **key**. Insertion starts with the second message.
- The "random prefix" in the login line is the gen-3 key. See `login-and-walk.md`.
- The 0x8088405 / 0x08088405 multiplier is the Borland `Random` LCG constant. 2.05 also uses it in its RTL `Random`. That is a coincidence of the toolchain, not a shared state.

2.05 send pipeline (update 2026-10-21) [2.05]

1. Build each message as GCHAR id + data + `\n`.
2. From the second message on (every message after the login line, including the rest of the first frame), apply the gen-3 insertion to **this message**, before compression. The byte may land at position 1, before the id.
3. Append it to the out buffer.
4. Flush: zlib-compress the whole buffer (several messages) into one block and prefix it with u16be length. A flush happens at 0x1000 bytes or at the end of the tick. The link is dropped if more than 0x40000 bytes are queued.

Receive is the reverse, with no cipher: u16be → inflate → split on `\n`, with the RAWDATA escape. The receive dispatcher handles 59 ids. Every other id, including 26 and 49–59, hits a silent default.

A separate UDP peer channel is not framed this way; see `udp-p2p.md`.

`protocol/cross-version-anchors.md`

Cross-version anchor search — 2171 / 222 / 231 vs v2.05

Date: 2026-10-14

Method

Searched the three versioned C exports for anchors known from format notes and the v2.05 corpus (`DispatchGs1.txt`, plate map, CRC/PNG/level strings).

Hits

Anchor	2171	222	231	Notes
`GLEVNW01`	4	3	0	Text level magic. 231 may use a different path or string form.
`GR-V1`	33	33	17	Binary level magic. 231 also checks `GR-V1.04`.
`IHDR`	1	1	1	PNG writer present in all three.
`BOARD`	1	1	0	Text board line.
`0xEDB88320`	0	0	0	Constant is lowercase `0xedb88320` in the export.

Identified clusters (2171)

Level load

- Level load-from-stream path — reads 8-byte magic, compares `"GLEVNW01"`, calls loader.
- Nearby stream-handling helpers sit beside that path.
- Earlier site — magic check chain for `GR-V1.00–03` and `GLEVNW01` (version/capability path).
- Writer site — emits `"GLEVNW01\n"`.

PNG

- `IHDR` compare inside the export path. Matches the earlier note that PNG writers still lack recovered names in the export.

CRC-32

- CRC-32 table builder — contains the `>> 1 ^ 0xedb88320` polynomial step.
- Nearby helpers are likely init / update.

Version notes

- 2171 and 222 are nearly identical in magic-check structure.
- 231 drops the plain `GLEVNW01` / `BOARD` string hits in this export and adds `GR-V1.04`. Level load may have been refactored or the string is built differently.
- CRC and PNG anchors are stable across all three.

Suggested names (2171, confirm in the local decompile project)

- Level load-from-stream → `Level_LoadFromStream` (or `TLevel_Load`)
- CRC table builder → `Crc32_BuildTable`
- PNG path containing `IHDR` → `Png_WriteChunk` / `Png_Export`

Next

Repeat the same enclosing-function lookup for 222 and 231, then rename inside the Ghidra projects. The v2.05 `DispatchGs1.txt` is opcode-oriented and does not share these level/CRC strings; use it for the script-dispatcher pass separately.

`protocol/crosscheck-captures-vs-research.md`

Cross-check: captures vs other-project research

Date: 2026-10-14

Codec (confirmed)

From `graalcodec.txt` / `PROTOCOL.md` and the gserver-research README:

```
wire_byte = logical_id + 0x20
length_byte = nbytes + 0x20
```

14-bit: `(b0-0x20)*0x80 + (b1-0x20)`
21-bit: `(b0-0x20)*0x4000 + (b1-0x20)*0x80 + (b2-0x20)`

Line: `(type+0x20) + payload + "\n"`
First line of a session inserts one extra LCG byte.

Framing: 16-bit big-endian length + zlib (or raw, or encrypted). Matches the pcaps.

Login (confirmed)

v2.05 shape from the research:

```
Chr((rand & 0x7F) + 0x20) + GNW##### + length-biased account/password
```

Pcap matches (credentials redacted): - `GNW28015` (session 01) - `GNW22122` (session 03) - `GNW03014` (session 04) - `GNW07080` (session 11)

The leading garbage byte is the random prefix. Account and password follow as length-biased fields.

Movement (now identified)

Short client frames in the clear sessions start with `"` (0x22).
`0x22 - 0x20 = 0x02`.

Research notes: after login, short frames headed `02` / `04` / `06` are walk and session.
So the `"1-\n` / `"/[\n` packets are **logical type 0x02** (walk), character-encoded, newline-terminated.

Payload after the type byte is still to be split (x/y/dir). The classic/precise position codecs remain property helpers, not this packet.

Encryption

Even modes XOR with the Borland LCG:

```
state = state * 0x08088405 + increment
dword ^= state
```

Initial state `0x04A80B38`. Explains why 14801 sessions stop decompressing after the first frame.

Scripting references promoted

- `commands.rtf` — classic GS1 command surface (control flow, NPC/player verbs, item indices, directions, baddy names, message codes).
- `Graal_Reborn_Scripting_Guide.rtf` — 2008 forum guide; same language plus extras (setbow, mouse flags, showtext, tiledef split).

Direction note from the rtf: 0–3, with `up/left/down/right`. The earlier client note used 0 south / 1 west / 2 east / 3 north. Treat the rtf order as the script-facing names; confirm the numeric mapping against a named decompile before locking Seihai.

Files

- `/shworkingpapers/protocol/from-other-projects/` — PROTOCOL.md, graalcodec.txt, players.md, flows.md, gserver-research README
- `/shworkingpapers/scripting/commands.rtf`
- `/shworkingpapers/scripting/Graal_Reborn_Scripting_Guide.rtf`

Update 2026-10-21

- "Even modes XOR with the Borland LCG" describes gs2lib gen 4/5 (2.19+ clients), keyed by the login key, and XORs only the first 4 or 12 dwords. 2.05 is gen 3 (single-byte insertion). See `codec-and-framing.md`.
- Direction numbering is settled: 0 up, 1 left, 2 down, 3 right (2.22 `commands.rtf`, GServer).
- The movement frame is `PLI_PLAYERPROPS`. See `login-and-walk.md`.

`protocol/file-transfer.md`

File transfer

Date: 2026-10-21

Sources: GServer-v2 `TPlayer::sendFile`, `msgPLI_WANTFILE`, `msgPLI_UPDATEFILE` (GPLv3, summarized); 2.05 dispatch and sender tables. Tags as elsewhere.

Requests (client → server)

Id	Name	Layout	2.05	Notes
23	WANTFILE	rest = filename	✓	Pre-2.1: the server appends `.gif` when there is no extension **[srv]**
34	UPDATEFILE	`i5 cached modtime` + rest = filename	✓	The server sends the file if the modtime differs; otherwise it replies "up to date" **[srv]**
47	VERIFYWANTSEND	`i5 crc32` + rest = filename	·	2.x later
155/156	RC_LARGEFILESTART/END	RC upload framing	·	RC only

Replies (server → client)

Id	Name	Layout	2.05
100	RAWDATA	`i3 length` of the next message	✓
102	FILE	see below; always behind RAWDATA	✓ via 100
30	FILESENDFAILED	rest = filename. **Pre-2.1 also uses this for "already up to date"**	✓
45	FILEUPTODATE	rest = filename (≥ 2.1)	✓ (handled, but the server does not send it to 2.05)
68 / 69	LARGEFILESTART / END	rest = filename	·

Id	Name	Layout	2.05
84	LARGEFILESIZE	`i5 size`	·

PLO_FILE body [srv]

Client	Layout
< 2.1 (**2.05**)	`c(102) str filename` + file bytes. No modtime, no trailing `\n`. RAWDATA length = 1 + 1 + name + data
2.1 – 2.13	`c(102) i5 modtime str filename` + bytes + `\n`, all in one piece
≥ 2.14	as above, split into 32000-byte chunks; files > 32000 bytes are wrapped in LARGEFILESTART / LARGEFILESIZE / LARGEFILEEND

Clients before 2.14 cannot receive files over 64000 bytes. The server answers FILESENDFAILED instead. **[srv]**

The layout follows the server's token table, not the client (2026-10-31)

Every version branch in GServer's file code tests `versionID`, and `versionID` comes only from the login token looked up in gs2lib's table. A token that is allowed (`allowedversions.txt`) but not in the table gets `CLVER_UNKNOWN` (0), which is below every real version. **[srv]**

What such a client gets, from GServer-v2 `efb5b90` (`TPlayer::sendFile`, `msgPLI_WANTFILE`, `msgPLI_UPDATEFILE`, `TPlayerProps`): **[srv]**

Below	Server behaviour
2.1	`PLO_FILE` without the GINT5 modtime and without the trailing `\n`
2.1	an up-to-date `UPDATEFILE` is answered with 30 (`FILESENDFAILED`), not 45
2.1	`.gif` is added to a requested name with no extension, and numbered head, body, sword and shield props get `.gif` names
2.1	the level goes through `sendLevel141`, with no `PLO_SETACTIVELEVEL` (156)
2.14	no large-file packets: a file over 64000 bytes is answered with 30

The 5.24 "PNG as file name" bug

- Stock gs2lib has no `G3D15010` (5.24r3, 5.24r4, 6.00 beta2) and no `G3D03048` (6.00 beta1), so GServer serves those builds every pre-2.1 path. Read in gs2lib `5c36ab8` (the version GServer beta4 fetches; see the table below). **[src]**
- The client always reads `PLO_FILE` as `i5 modtime` + `str filename` + data. Given the old layout, it takes the length byte and the first 4 characters of the name as the modtime, and the 5th character as the name length. **[srv][inf]**
- For `sword1.png` that is `*swor` as the modtime and `d` (68) as the length, so the "name" is `1.png` followed by 63 bytes of PNG data. That is the lab message "Couldn't open …1.png‰PNG… for writing". **[cap]** for the message, the byte walk is worked from the server code.
- With a gs2lib copy that knows `G3D15010`, the same server sent 5.24r4 `&w1vH)state.png‰PNG…` (GINT5 modtime, then the name) and the client saved the files normally (v5x lab proxy log 2026-10-30b, connection c6). **[cap]**

gs2lib's version table [src]

Read in gs2lib `5c36ab8` (`src/IUtil.cpp` `clientVersions[]`, `include/IUtil.h` `CLVER_*`):

- `getVersionID` walks `clientVersions[]` and returns the **index** of the first exact match, or `CLVER_UNKNOWN` (0). The index is the `CLVER_*` value. So a token's rank is simply its position in the array, and every `versionID < CLVER_x` test in GServer compares positions.
- The 5.x/6.x part of the table, in order:

Token	Enum	Build
`G3D22067`	`CLVER_5_07`	5.00r7
`G3D14097`	`CLVER_5_12`	5.12
`G3D26090`	`CLVER_5_31x`	5.30r3, 5.31r4, 6.00r2, 6.00r4
`G3D3007A`	`CLVER_6_015`	6.01x (no lab build)
`G3D2505C`	`CLVER_6_034`	6.034/6.035
`G3D0311C`	`CLVER_6_037`	6.03r7 Windows
`G3D0511C`	`CLVER_6_037_LINUX`	6.03r7 Linux

- `G3D15010` and `G3D03048` appear nowhere in the source. (With `gs2lib-CLVER_5_24.patch`, `G3D15010` becomes `CLVER_5_24`, between `G3D14097` and `G3D26090`.) All 5.x/6.x entries sit far above `CLVER_2_14`.
- The array and the enum must stay in step: an entry added to one without the other shifts every later version id.

Rule. A server for 5.x and 6.x clients must know every token it allows. Every 5.x/6.x slot in the table is above `CLVER_2_14`, so any of them gives files with the modtime and large files in chunks. **[src]** The lab got there by renaming one entry to `G3D15010` (`G3D26090` in the v5 lab, `G3D14097` in the "Play v5 and v6 clients" lab, see `version-profiles/lab-v5-2026-10-30.md` and `v5v6-lab-play.md`). Each rename takes the renamed build off the table (5.30 to 6.00r4, or 5.12). The clean fix is a new `CLVER_5_24` between `CLVER_5_12` and `CLVER_5_31x`, added to both the array and the enum. **Done 2026-10-31:** `tools/lab-servers/gs2lib-CLVER_5_24.patch` (and a `b839106` version for the efb5b90 lab build) adds that slot. Real 5.24r4 on GServer efb5b90 with the patched library got every file in the new layout, including `pics1.png` as a large file, and saved them byte for byte. On stock gs2lib it reproduced the bug. **[cap]** Details are in `tools/lab-servers/README.md`. **[src]** for the table, **[inf]** for the placement: it follows the build dates, and no original server's table was seen. Where 5.24 sits matters only for tests against `CLVER_5_*` or `CLVER_6_015`; GServer `efb5b90` has one (`versionID < CLVER_6_015` in `TPlayerLogin.cpp`), and 5.24 is below `CLVER_6_015` in either slot, so it takes the same branch.

Client side. A client cannot tell the server which layout to use. Seihai reads with the profile's layout first and, if the name comes out with control bytes, no dot, or longer than the message, reads the other layout and keeps it for the connection (Seihai `documentation/file-cache.md`). **[inf]**

What 5.x asks for at login [cap]

- 5.24r4, 5.30r3 and 6.03r7 open with `basepackage.gupd`. With an empty cache it goes as 23 (`WANTFILE`), twice (v5x lab 2026-10-30b; `v5v6-windows-unpacked.md`).
- In the "Play v5 and v6 clients" runs, 5.24 and 6.03r7 sent it as **47** (`VERIFYWANTSEND`, `i5 crc32` + name), twice, while the `G3D26090` build sent 23 (`lab-v5v6/logs`). That 47 is used when a copy is already cached is **[inf]**: the cache state of those runs was not recorded.
- 5.00r7 settles it (lab, 2026-10-31, stock gs2lib): in one run the real client sent `basepackage.gupd` as **47** on the Login connection and as **23** after the 178 warp to "Lab Server". Its install holds `updatepackages/`

`basepackage.gupd` (the Login server's copy) but no `updatepackages/Lab%032Server/`. So 47 goes out when that server's copy is cached, and 23 when it is not; the cache is per server name. **[cap]**
- The same run checked the layout on the stock library: `G3D22067` is entry 49 of `clientVersions[]` in the shipped `libgs2lib.so` (`b839106`), named `5.07`. The server sent a small file as `102` with the GINT5 modtime, and a 541,248-byte file as `68`, `84`, `102` chunks each with modtime and name, then `69`. The real 5.00r7 saved it byte for byte. **[cap][bin]** Write-up: Seihai `documentation/v5v6-profiles.md` (seihai#293).
- Then `UPDATEFILE` for `idle.gani`, `sprites.png`, `body.png`, `head0.png`, `shield1.png` and the `guigames_*` images, and `WANTFILE` for the fonts (`arial.ttf`, `arialbd.ttf`) and `tiledefs<server>.txt`.

Typical flow for 2.05 [srv][inf]

1. A level arrives that references `head25.png`, `sword1.png`, or `walk.gani`.
2. The client checks its local cache. For a cached file it sends `UPDATEFILE(modtime, name)`, otherwise `WANTFILE(name)`.
3. The server replies `RAWDATA(len)` + `FILE(name, bytes)`, or `FILESENDFAILED(name)`, which for 2.05 means "missing or unchanged".

The 2026-10-04 clear sessions show this pattern. Server frames there contain `sword1.png`, `head25.png`, `walk.gani`, and `steps.wav` pushes. **[cap]**

Upload (RC only)

RemoteControl's folder change (0x5a wire) and upload ids are in `tools/remotecontrol.md`. In gs2lib, the RC file browser covers ids 89–98 (`PLI_RC_FILEBROWSER_*`) and `PLO_RC_FILEBROWSER_DIRLIST`/`DIR`/`MESSAGE` (65–67). Game clients cannot upload. **[srv]**

Open

- How 2.05 picks its local cache folder, and whether it ever sends `UPDATEFILE` for level files or only for images.
- The 2.05 handlers for 30 and 45 were found in the jump table but have not been traced.

`protocol/format-notes-from-salvage.md`

Format notes from salvage (2026-10-14)

Promoted from `/recycler/salvage/seihai-original-data/docs/` and dropin formats.

Level formats

GLEVNW01 (.nw)

- First line: `GLEVNW01`
- `BOARD x y width layer tilesdata` — horizontal run, 12-bit tile indices packed as two base64 chars
- Alphabet: `A-Za-z0-9+/` (AA = 0, // = 4095)
- Stored at `levelTiles[x + y*64]`; layer ignored
- Other records: LINK, SIGN/SIGNEND, NPC/NPCEND, BADDY/BADDYEND, CHEST

- Loader also accepts GR-V1.01–03 (binary .graal) and Z3-V1.03/04 (Zelda)

Board model

- 64×64 cells, 16 px each, world = 1024×1024 px
- File pairs: `.seihai`/`.graal`, `.as`/`.nw`
- Headers: SH-V1.00–03, GR-V1.00–03, GLEVAS01, GLEVNW01, Z3-V1.03/04

GRMAP001 (.gmap)

- Level list with HEIGHTS blocks, not a board itself

Collision (tiletypes)

- Client does not inspect the tileset image
- Board stores tile id (0–4095); collision is a separate 4096-byte table
- `isOnWall`: type ≥ 20
- `isOnWater`: type == 11
- Table lives in `tiletypes.h` / `tiledefs.txt` (128 values per line)

Sign text

- Binary sign record: `byte0 = x+0x20`, `byte1 = y+0x20`, rest = sign alphabet
- Do not apply binary decoder to GLEVNW01 signs (already text) or NPC scripts (Latin-1, 0xA7 = newline)
- Drop-in decoder: `formats/sign-decoder/signText.js`

Files promoted

- `/shworkingpapers/assets/formats/` — glevnw01, grmap001, gani, tiletypes, boards, systems
- `/shworkingpapers/assets/tiledef/tiledefs.txt`
- `/shworkingpapers/scripting/sign-decoder.md`
- `/shdecompiling/graal-v205/` — Graal.exe.c, .gzf, DispatchGs1.txt, PlateGs1.java (do not release)

`protocol/graphics-movement-bytecode.md`

Graphics, movement, and bytecode research notes

Date: 2026-10-14
Sources: GServer-v2 `TLevel.cpp` / `TPlayer.cpp` / `TNPC.cpp`, OpenGS1 VM.

Graphics pushes

Board data

- `PLO_BOARDPACKET`: raw 64×64 `short` tile array (layer 0), 8192 bytes + newline.
- `PLO_BOARDLAYER`: `(layer:u8)(0)(0)(64)(64)` + raw `short[4096]` for that layer.
- `PLO_LEVELBOARD` / `PLO_BOARDMODIFY`: concatenated `getBoardStr()` results from `TLevelBoardChange`.
    - Change holds `x, y, width, height, tiles, oldTiles, modTime`.

    - Byte layout confirmed on the wire for 2.05: `c x, c y, c w, c h` + w × h GSHORT tiles, one change per message. See `packet-layouts.md`, "Board change on the wire". **[cap]**

Other level objects

- `PLO_NPCPROPS` + optional `PLO_NPCBYTECODE` (wrapped in `PLO_RAWDATA` for clients ≥ 4.0211).
- `PLO_LEVELLINK`, `PLO_LEVELSIGN`, `PLO_LEVELCHEST`, `PLO_HORSEADD`, `PLO_BADDYPROPS`, `PLO_ITEMDEL`.

GANI

- Client maintains a fixed list of standard GANIs (`walk.gani`, `sword.gani`, `idle.gani`, etc.).
- Animation state is part of player/NPC props, not a separate board packet.

Player movement

- Pre-2.3 clients: server watches `gr.x` / `gr.y` flags and emits `PLPROP_X` / `PLPROP_Y` (position × 2) via a queued `-gr_movement` path.
- ≥ 2.3 clients: different (direct) path; the `gr.*` workaround is skipped.
- Positions are floats internally; on-wire often scaled by 2 and sent as chars/shorts.
- Client is expected to send local movement; server is authoritative for other players.

Still needed: exact packet ID and field layout the *client* emits for its own movement (direction, position, GANI).

Scripting — GS1 bytecode

OpenGS1 defines a stack-based bytecode VM.

Bytecode file layout (`Bytecode.cpp`)

1. `u32` body offset
2. `u32` string-constant count + length-prefixed strings
3. `u32` number-constant count + `f32`s
4. Bytecode body (starts at body offset)

PackedValue (4 bytes)

- `valueType:4` (`PACKVALUE_CONST_NUMBER`, `CONST_STRING`, `CONST_ARRAY`, `NAMED`)
- `value:16` (index into constant table or name table)
- `unused:12`

Opcodes (from `Operation.hpp`)

Opcode	Meaning
`OP_PUSH`	Push PackedValue
`OP_ASSIGN`	S(1) = S(0)
`OP_ARR_SET` / `OP_ARR_GET`	Array access
`OP_ADD` … `OP_POW`	Arithmetic
`OP_INC` / `OP_INCPUSH` / `OP_DEC` / `OP_DECPUSH`	Inc/dec
`OP_CALL` / `OP_CMD_CALL`	Function / command call
`OP_JMP` / `OP_JAL` / `OP_RET`	Control flow

Opcode	Meaning
`OP_EQ` / `OP_LT` / `OP_GT` / `OP_LTE` / `OP_GTE`	Comparisons
`OP_NOT` / `OP_AND` / `OP_OR`	Logic
`OP_JEZ` / `OP_JNZ`	Conditional jumps
`OP_STOP`	Halt
`OP_DBG_OUT`	Debug

On-wire

- `PLO_NPCBYTECODE` carries the raw bytecode blob for an NPC.
- Sent only to clients ≥ `CLVER_4_0211` and only when bytecode is non-empty.
- Wrapped in `PLO_RAWDATA` with a length prefix.

What is already usable for Seihai

- Board packet sizes and IDs are known.
- Full GS1 opcode set and constant-table layout are known.
- Movement scaling (×2) and the pre-2.3 `gr.x`/`gr.y` path are known.

What still needs capture or confirmation

1. ~~Exact byte layout of `TLevelBoardChange::getBoardStr()`.~~ Done: see `packet-layouts.md`.
2. Client→server movement packet (ID + fields) for both <2.3 and ≥2.3.
3. How the original Delphi client stores/receives `PLO_NPCBYTECODE` (Ghidra trace).
4. Mapping from GS1 command names (`setcharprop`, `showcharacter`, etc.) to host functions the VM calls.
5. Whether later clients ever use a different bytecode format (GS2) or only V8.

`protocol/gserv-1999.md`

GSERV-token protocol, 1.21 to 1.32 (May 1999 to January 2000)

Date: 2026-10-21

This covers the clients from 1.21 to 1.32 and the GServer and RemoteControl builds shipped with 1.21, 1.28 beta 2 and 1.3 test. It is the first central-server protocol: accounts, a login token, and zlib on every frame. In the GServer-v2 numbering this is encryption generation 2, the same one 1.33 to 1.39 use with GNW tokens. The notes come from static analysis of the shipped executables. No decompiler output is reproduced.

Gen 1 to gen 2 in one table

	Gen 1 (1st Graal to 1.1.9)	Gen 2 (1.21 on)
Server	Lobby/relay, no game logic	GServer: accounts, levels, NPCs, flags, weapons
Port	14801 local default, 14802 Cyberjoueur	14804 (`<list-host-classic>`) for 1.21 to 1.3 test, 14900 from 1.31
Framing	Text lines	`u16` big-endian length, then a zlib stream
Message ids	One letter (`D` plus a type letter)	`chr(id+32)` as the first byte of each line
Identity	Server-given 8-character id, nickname sent as data	Account and password at login

	Gen 1 (1st Graal to 1.1.9)	Gen 2 (1.21 on)
Levels	HTTP from the level server	Sent by the GServer

Nothing in gen 2 is encrypted. The DES code in these clients is only used for local files (see `formats/graalsav.md`).

Framing

Sending: messages queue in one buffer as `chr(id+32) + data + "\n"`. On flush the whole buffer is deflated with zlib 1.0.4 and written as:

```
[len_hi][len_lo][zlib stream of len bytes]
```

There is no size threshold. Every flush is compressed.

Receiving: frames are read the same way, inflated, and split on `\n`. Server id 100 (`chr(132)`) puts the reader into a raw mode that takes a given byte count instead of a line, for file data. `nodecompressionerror=` in `graalconfig.txt` (from 1.28 beta 2) hides the inflate error message.

Login

The first message after `connect` is a normal id-0 line:

```
' '                    type: chr(0+32) = player. The GServer treats '!' (type 1) as RemoteControl
"GSERVnnn"             8 bytes
chr(len(account)+32) account
chr(len(password)+32) password
```

The connect dialog's defaults are `guest` / `guest`. This is the same shape the later gen-3 login uses, minus the key byte and with GSERV instead of GNW.

Client	Token
1.21, RemoteControl 1.21	`GSERV007`
RemoteControl 1.28, RemoteControl 1.3	`GSERV011`
1.28 beta 2	`GSERV013`
1.3 test	`GSERV014`
1.31	`GSERV016`
1.32	`GSERV018`

The tokens are a protocol revision and not a date. GNW tokens, which are dates, start with 1.33.

What the GServers accept

The server compares the token against a fixed list and stores the matching number (7 to 14) on the connection. Then it checks the minimum for the connection type.

GServer	Accepts	Player minimum and refusal text
1.21 (May 1999)	`GSERV007`	"You need Graal version 1.2 rev1 to play on this server."
1.28 (Nov 1999)	`GSERV007` to `GSERV013`	13: "You need at least Graal version 1.28beta2 to play on this server."

GServer	Accepts	Player minimum and refusal text
1.3 (Nov 1999)	`GSERV007` to `GSERV014`	14: "You need at least Graal version 1.3 to play on this server."

The older tokens stay on the list so that RemoteControl (GSERV011) can still log in. Refusals and login errors ("The account has been disabled by an admin.", "The account is already in use.", "An account with the name and password you typed in doesn't exist.") go out as server id 16, which is DISCMESSAGE in the later numbering. Captured under Wine (`client-internals/live-trace-early.md`): after a good player login GServer13 sends 9 (own props), 28 (flags, only if the account has any), 14 (warp), 100 followed by a separate frame holding 101 and the board, 6 (level name), 1 (links), 5 (signs), 39 (level modification time), 3 (NPC props with scripts), and 10. GServer 1.28 sends the same sequence without 39, which arrives with the 1.3 level cache. GServer 1.21 also sends no 39, and it packs the whole reply (9, 14, 100, 101 with the board, 6, 1, 10) into a single zlib frame. When another player is already on the level, the newcomer gets an 8 (other player's props) right after its own 9. A token newer than the server's list is refused like an old one: GServer13 answers GSERV016 (1.31) and GSERV018 (1.32) with id 16 "You need at least Graal version 1.3 to play on this server." **[cap]**

Message ids

The server-to-client numbering used through 2.x is already in place in 1.21. The 1.21 handlers line up with the later names: 1 links (`TEditorLink`), 2 baddy props (`TComputer`), 3 NPC props (`TEditorNPC`), 4 chest, 5 sign (`TAdventureSign`), 7 board modify, 8 other player props (`TOtherClient`), 21 throw carried (`TAdventureCarry`), 22 item add (`fullheart`, `sword`, `shield`), 23 item delete (`extra.wav`).

Client	Server-to-client ids handled
1.21	0 to 34 except 30, plus 100
1.28 beta 2	0 to 38, plus 100
1.3 test	0 to 39, plus 100
1.31	0 to 40, plus 100
1.32	0 to 41, plus 100

The client-to-server ids visible as constants are 0 and 10 to 26 in 1.21, and 0 and 10 to 30 in 1.32. Other sends pass the id in a register, so these lists are not complete.

Props by build

	1.21	1.28 b2	1.3 test	1.31	1.32
Player props the client parses	0–24	0–31	0–32	0–32	0–33
Player props the client sends	0–24	0–24, 26, 31	0–24, 26, 31, 32	same as 1.3	0–24, 26, 31–33
NPC props	0–17	0–32	0–34	0–34	0–34
Baddy props	0–10	0–10	0–10	0–10	0–10

The prop numbers grow at the end, so a decoder can use the later tables (`player-props.md`) and stop at the build's last id. Per-build values, and the client-to-server ids, are in `client-internals/version-profiles/v121-v132.toml`.

UDP (1.28 beta 2 on)

The 1.28 beta 2 notes call this "direct data packages", and it is the first build that links `bind`, `sendto` and `recvfrom`. The client binds a UDP socket and sends single, uncompressed, unframed messages (anything without

a `\n`) straight to another player's address and port. In 1.28 beta 2 captures, a walking client sent id 0 to the other player every 100 ms, while the server got only the changed props over TCP and forwarded nothing. Id 0 carries X, Y, SPRITE, STATUS and CARRYSPRITE (props 15 to 19; `0x1F` is −1, nothing carried), the same fields that 2.05 sends in UDP id 0. Id 12 is arrow add, with the same 5 bytes as relay `A`. One player per level sends the level NPCs' arrows to the others: the level leader, picked by the server with id 10 (`ISLEADER`, empty). The server sends 10 at the end of the first player's login reply, and to the next player when the leader leaves. **[cap]** The address and port are stored on the other player's record, so they most likely come from the server (*inferred*). The main loop reads TCP and UDP separately ("tcp read", "udp read" in the 1.32 profiler labels). 1.31 adds `noudp=` and `fixedudpport=` to `graalconfig.txt` (the catalogue gives 14899 as the fixed-port default). The client handles UDP ids 0 and 10 to 22 in 1.28 beta 2 and 1.3 test, and 0 and 10 to 23 from 1.31. These are the same ids, with the same +32 offset, that 1.33 handles. Ids 0 (movement) and 12 (arrow add) match 2.05 on the wire. A newcomer sends one empty id 22 to each player already in the level; 2.05 uses 22 for chat. The other ids have not been mapped yet. `protocol/udp-p2p.md` covers the 2.05 UDP ids and is the closest reference.

Open questions

- The meanings of ids 35 to 41 and the full client-to-server list. A diff against the 1.39 Dhrake names would settle most of them.
- Whether RemoteControl uses the same framing. It probably does, since the server code path is shared, but it hasn't been checked.

`protocol/lab-checks-2026-10-24.md`

Lab checks, 2026-10-24

Derived facts from real-client runs against a local lab only. No public server was contacted. Placeholder accounts. Captures stay off the repo.

Tampercheck (client PLI 39)

Every build sends the same 10 raw bytes at every login, regardless of account and login key. It is a per-build constant, not two GINT5 words (bytes below 0x20 occur; GINT5 bytes cannot be below 0x20). No byte is 0x0a, so it survives line framing unescaped.

Build	Token	Constant
2.17	`GNW22122`	`2a7bad195e231243d82c`
2.22	`GNW03014`	`250e4d0f962b03ede631`
2.31	`GNW28015`	`291655b7602e87c9eb82`

Position: second message of the first client frame, right after PLI 37 (language).

Re-send: - 2.17: once, at login. - 2.22: at login, and again once for every gmap-edge crossing, and after every client-initiated level-link request (a later lab run; not after a server-side warp). No periodic send. - 2.31: at login, then about every 3.2 s. 2.31 also sends PLI 24 (1 byte, value 9) about every 5.1 s, and PLI 46 (1 byte, value 1), which a later lab run showed is sent on every level entry, not only at login (`client-internals/decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`; also on doors, links and server warps, `client-internals/decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md`). 46 is 2.31-only; a later lab run saw 2.22 send 24 too, but only while another player was in its level.

GServer-v2 accepted all three and did not log an invalid packet.

How the 10 bytes are built (two chained CRC-32 words over the client's own memory image) and why 2.31 repeats them every 3.2 s: `client-internals/decomp-findings-2026-10-24-b/01-tamper-check-id39.md` and `client-internals/decomp-findings-2026-10-24-b/03-periodic-ids-2.31.md`. The two words per build are in `version-profiles/tampercheck-words.tsv`.

2.17 listserver login

One TCP connection to port 14922. Plain zlib framing (u16be length + zlib body). No gen-5, no version token.

Client sends one frame holding two messages: `GCHAR 0` + `newmain`, then `GCHAR 1` + GSTRING account + GSTRING password.

Server answers with two plain zlib frames: frame 1 = id 0 server list; frame 2 = id 2 status text, id 3 site URL, id 5 donation URL. The client then closes.

After the list, the gserver login is type 0, key, `GNW22122`, account, password, then gen-3. The junk-byte rule decoded 100% of the 2.17 messages.

2.22 gmap edge crossing (2x2 gmap)

Crossing uses only normal props. No level-warp packet.

- A client flag `gr.x` / `gr.y` is set on every half-tile step (PLI 18). It carries the global position (local + 64 × cell), so it runs past 63.5.
- When the global position wraps, the same props message carries the wrapped local coordinate and the cell prop (GMAPLEVELX = prop 43, GMAPLEVELY = prop 44).
- East: at global x 64.0 the client sends PLI 39 (tamper constant), `gr.x=64`, and props with local X = 0 and prop 43 = 1.
- South: the frame at global y 64.0 carries PLI 39 and the flag; the props with local Y = 0.5 and prop 44 = 1 follow in the next half-step frame (position props go out only on whole-tile steps).
- West and north follow the same pattern with the wrapped coordinate and the cell prop set back to 0.
- Rule: local = global − 64 × cell. The cell prop changes in the same props message as the wrapped coordinate.

Server answer to each crossing (one frame): id 6 new level name, id 39 level mod time (only when not cached), id 0 empty board, id 6 gmap name, 174, 10, 156 gmap name. A second visit omits the server's 39.

Login on a gmap: server sends PLO_PLAYERWARP2 (49) with x×2, y×2, z+50, gmap cell x, gmap cell y, then the gmap name. It follows with PLO_LEVELNAME (6) = the level, then 6 = the gmap file name. The client asks for the gmap file. The first big server frame also carries the neighbour levels' boards. The client sends PLI_ADJACENTLEVEL (35) at login for the three neighbour levels (GINT5 modtime + name; modtime is 0 on a cold cache).

Buddy, ignore, mute (2.22)

- Buddy add and delete: no network traffic. The client writes and deletes `buddyaccounts.txt` locally (deletes the file when empty). `verifybuddies` / `addbuddy` / `deletebuddy` were not sent in this lab, so the listserver-side token layout is still unseen.
- Ignore: local too (`ignoredaccounts.txt`, one account per line). No packet.
- Mute: PLI 43 with GSHORT playerId + GCHAR 1 (mute) or 0 (unmute). One message per click. GServer-v2 has no handler (falls into the invalid-packet counter), but two sends did not disconnect.

- Player-list window sections: Staff, Buddies, Guild, Normal, Ignore. Right-click: Message, History, Buddy, Ignore, Mute, Profile.

Gen-5 XOR confirmation

Client-originated frames in the 2.22 sessions: ctype 0x02 uncompressed (580) and ctype 0x04 zlib (17). No ctype 0x06 (bz2) was sent; the bz2 path stays unseen (needs a send over 0x1000 bytes).

The 17 zlib frames have compressed bodies whose lengths mod 4 are 1, 2 or 3, never 0. All 17 decode and decompress with the one-step-per-group rule (partial last group steps, cap 4 for zlib). The old floor rule fails on them. So the corrected rule matches a real client, including short frames.

Missing binaries

No 2.1.x (2.1 through 2.16) or 2.3 binaries are recoverable. Those rows stay inferred from neighbours and release notes. Practical treatment: alias 2.1–2.16 to 2.05 or 2.17, and 2.3 to 2.31.

`protocol/listserver-login-type-matrix-2026-10-25.md`

Public listserver: the reply depends on the login type (matrix, 2026-10-25)

Derived note. The data is one run of the read-only list check made by the user on their own Linux machine on 2026-10-25 (started 22:43 JST): 19 Seihai version profiles, each tried once on port 14922 and once on port 14923 of `<list-host-reborn>`, one list-only login per run (no game-server login, no chat, no warp), 38 runs in all. Each run was captured with `tcpdump`. The account name is masked, and no login frame bytes, credentials or keys appear here; the captures stay private. Per-run table: `listserver-login-type-matrix-2026-10-25.tsv`.

This extends `public-listserver-2026-10-25.md` (first two runs: 2.00, 2.05, 2.17, 2.22, 2.31) to every profile that has a listserver.

Result

The listserver chooses its reply by the **login type** the client sends, not by the version number or the token.

login sent	profiles in the matrix	port 14922	port 14923
classic: one plain zlib frame holding id 0 `newmain` and the id 1 account and password message (36 bytes of TCP payload)	1.41, 1.42, 2.00, 2.01, 2.02, 2.02 rev 2, 2.03, 2.04, 2.1, 2.12, 2.13, 2.14, 2.15, 2.16, 2.17 (15 runs per port)	connection accepted, **no reply** for the 5 s read timeout (5.3 to 5.8 s per run) in all 15	**legacy list** in all 15: one plain zlib id 0 frame, 2 or 4 rows, no status, site or upgrade lines
gen-5: id 7 with the key byte and token `GNW30123` and `newmain` in one plain zlib frame (28 bytes of TCP payload), then a gen-5 frame with id 1 (20 bytes)	2.05 (see below), 2.22, 2.3, 2.31	**modern list** in all 4: 8 rows plus status, site and upgrade lines	**modern list** in all 4: the legacy list first (2.05, 2.22, 2.3: 12 rows in all; 2.31: 17 rows), then the status, site and upgrade lines

So 14923 answers both login types, and 14922 answers only the gen-5 login. For a default port that gets a list: classic login to 14923, gen-5 login to 14922 (14923 also works for it).

- The version number and the token do not decide it. 1.41 to 2.17 differ in version and token, and all got the same reply to the same login type. 2.17 (`GNW22122`) was silent on 14922 like 1.41.
- The silent case keeps the connection open; nothing is sent, and nothing closes it before the client's timeout. A classic login on 14922 is never refused, just unanswered.

- The row counts of the legacy list varied between runs (2 rows in 4 classic runs, 4 rows in 11) without following version order, and the 14923 modern list had 12 rows in three runs and 17 in one. The cause was not tested; the list changes between runs, not with the version.
- This fits the 2001 server tools' `settings.ini` (`tools/server-tools-2001.md`): clients connect on 14922 and older clients on 14923 (`clientPortOld`). The public service today answers the classic login only on 14923; the 2001 setting does not say what the current service does for each login type, so that is a reading of the two, not something tested.
- 14921 was not part of this run. It was refused at once in the earlier run.

What this does not show

- A real client of any of these versions was not captured. Every login here was sent by the Seihai reimplementation for the named profile; the result tells which login type the service answers on which port, and says nothing about what a real 1.41 to 2.17 client sends. Whether a real 2.17 client sends the classic frame is still unobserved.
- For 2.05, the binary that made the runs sent the **gen-5** login (28-byte frame, same shape as 2.22's), while the current Seihai main sends the classic login for the 2.05 profile. The 2.05 rows above are therefore the gen-5 case. A classic 2.05 login was not run; by the table above it would be answered on 14923 only.
- No game-server login was made, so nothing here says anything about game-server traffic.

Defaults that follow

`client-internals/version-profiles/` now gives the profiles the port of the login type the Seihai profile sends: 14923 for 1.41 to 2.17 (classic), 14922 for 2.22, 2.3 and 2.31 (gen-5). The ports are marked `confirmed` for every profile whose own login type was run on its port: the 15 classic profiles on 14923 and 2.22, 2.3 and 2.31 on 14922. 2.05 is `assumed` on 14923, because its classic login was not run. `list-ports.md` has the table.

`protocol/listserver.md`

Listserver protocol

Date: 2026-10-21

Sources: graal-serverlist (xtjoeytx, 2019) `TPlayer`, `TServer`, `main.cpp`; gs2lib `IEnums.h` SVO/SVI enums; 2.05 `TServerListClient` class. Tags: **[srv]** · **[2.05]** · **[cap]** · **[inf]**.

Two separate links exist:

1. **Client ↔ listserver**: login and the server list. This is what the game client talks to.
2. **Game server ↔ listserver**: registration, account verification, and profiles (SVO_*/SVI*_). Only servers use it.

Client ↔ listserver

Framing depends on the client generation. **[srv]**

Gen	Detected by	Frame
pre-2.2 (2.05)	default	`u16be len` + zlib. No cipher
2.2	id 4 `V2VER`	`u16be len` + bz2, partial XOR (limit 4)

Gen	Detected by	Frame
post-2.2	id 5 or 7	`u16be len` + compression byte + body, partial XOR

Client → listserver

Id	Name	Layout
0	V1VER	rest = version string (e.g. `newmain`)
1	SERVERLIST	`str account`, `str password`
4	V2VER	`c key` + `newmain` (selects 2.2 bz2 mode)
5	V2SERVERLISTRC	`c key` (RC)
7	V2ENCRYPTKEYCL	`c key` + `GNW30123newmain` (2.22 and 2.31 clients) **[bin]**
223	GRSECURELOGIN	Graal Reborn extension

Listserver → client

Id	Name	Layout
0	SVRLIST	`c count`, then per server: `c 8` (field count) and 8 GSTRINGs: `type+name`, language, description, url, version, player count, ip, port
1	NULL	—
2	STATUS	text (`\r` = line break)
3	SITEURL	url
4	ERROR	text; the connection closes
5	UPGURL	donate / upgrade url

The `type` prefix on a server name comes from the server's HQ level: `3` 3D, `P` gold, `H` bronze (hosted), `U` hidden, empty for silver. Pre-2.2 clients get no prefix for bronze. **[srv]**

2.05 specifics [2.05]

- `TServerListClient` derives from the same `TGraalClient` base as the game connection. It overrides only the receive handler. Its **send path is the base one, with no gen-3 insertion**, which fits a cipher-free pre-2.2 list link.
- Its receive handler switches on two ids only: 0 (server list: count followed by length-prefixed fields) and 1.
- The list host and port are not in the string table as plain text. The host is a DES-encrypted string that the server-list form builds one character at a time, together with its key. It decrypts to `<list-host-classic>`. The port is `14921`, and the first packet is id 0 with `newmain`. See `client-internals/des-and-hidden-strings.md`. **[2.05]**
- The lab run confirms the whole exchange (`tools/lab-setup.md`). **[cap]**
    1. The client sends id 0 `newmain` as soon as the list window opens.
    2. A listserver that answers with id 0 at that point fills the list. The client does not wait for id 1.
    3. The client closes the socket about two seconds after it gets the list.
    4. The field order shown in the window is name, language, description, url, version, players, ip, port, which matches the table above.
- 2.00 uses the same built-in, DES-hidden host and port as 2.05 and does not read `arrays2.dat`. In the lab a real 2.00 client sent one plain zlib frame, `GCHAR 0` + `newmain` + LF (no account or password), made no second listserver connection, and accepted a 2.17-style two-frame list reply, but that reply was synthetic (a lab-only forwarder translated the request), so what a real 2.00-era listserver sends is still unobserved and the repo's "2.05-style" list format stays unverified for 2.00 (`client-internals/decomp-`

findings-2026-10-24-b/07-lab-findings-46-186-200.md) (client-internals/decomp-findings-2026-10-24-b/04-v2.00-client.md).
- To point a stock 2.05 at another listserver, redirect <list-host-classic> in the hosts file and listen on 14921. Patching the encrypted string in memory also works.
- The captures show later listservers on ports 14922/14923 (GNW30123 login, GNW07080 guest). **[cap]** On the public service the reply depends on the login type (matrix of 2026-10-25, listserver-login-type-matrix-2026-10-25.md): a classic login gets a list on 14923 and no reply on 14922; a gen-5 login gets a list on both. **[cap]**

2.22 specifics [2.22][cap]

- The host list is not built in. It comes from arrays2.dat in the game folder, in clear text, checked against arrays2.sig: line 0 <list-host-reborn>, line 1 port 14922, lines 5 and 6 the fallbacks <list-host-graalcc> and <list-host-graalbiz>. They are tried in that order until one connects. Once loaded, the client keeps them in memory under the one-byte transform described in client-internals/des-and-hidden-strings.md, which is why a plain memory search misses them. The same file layout applies from 2.17 on (see client-versions.md and the 2.17+ section below).
- First packet: id 7 V2ENCRYPTKEYCL = GCHAR key + GNW30123 + newmain. This frame is plain zlib. Every later frame in both directions is post-2.2 framing (compression byte + partial XOR) keyed with that key.
- Then id 1 SERVERLIST = account + password. The list reply is the same id 0 layout as for 2.05.

GNW30123 (seen on 14922) is the 2.22/2.31 listserver token. Both clients send listserver id 7 as c key + GNW30123 + newmain, then switch to gen 5 and send id 1 with the account and password. It is a listserver protocol token, not a game build, so the game-version tables do not list it. **[bin]** The lab confirmed this for 2.22. **[cap]** (See client-versions.md.)

2.17+ list host [bin]

From 2.17 the list host and port are not built in. They are read from arrays2.dat (line 0 host, line 1 port, lines 5 and 6 fallback hosts), which must match the signature in arrays2.sig. 2.17 still logs in pre-2.2 style (id 0 newmain, then id 1) with no cipher.

2.17 flow, seen from a real client in the lab [cap]: one TCP connection to port 14922 (host and port read from arrays2.dat), plain zlib framing (u16be len + zlib body), no gen-5, no version token. The client sends **one frame holding two messages**: GCHAR 0 + newmain, then GCHAR 1 + GSTRING account + GSTRING password. The server answers with **two plain zlib frames**: frame 1 is id 0 (the server list); frame 2 holds id 2 (status text), id 3 (site URL) and id 5 (donation URL). The client then closes the connection (FIN). Compare 2.22 and 2.31: they open with GCHAR 7 + key + GNW30123 + newmain and use gen-5 for everything after, with the same list and status ids (0, 2, 3, 5). After the list, the 2.17 game-server login is type 0, key, GNW22122, account, password, then gen-3 framing; the junk-byte rule in codec-and-framing.md decoded every 2.17 message in the capture.

Game server ↔ listserver (summary) [srv]

Server → list (SVO_*, 0–31): set name, description, language, version, url, ip, port, player count; verify account (VERIACC2) and guild; get/set profile; player add/remove; ping; file get (v3); serverHQ password and level; request list, server info, and buddies; PM a player; register v3; send text.

List → server (SVI_*): verify-account results (VERIACC2 11), guild verification, file start/data/end (v3 15–17), profile, error message, server info, request/send text (19/20), PM player (29), assign PC id (30), ping (99), rawdata (100).

Only needed if Seihai ever hosts its own listserver. The client only needs ids 0–5 above.

Per-generation login frames, real clients against Seihai's frames, and the open question of why the public 14922 ignores a classic login: `login-frames-by-version-2026-10-25.md`.

`protocol/login-and-walk.md`

Login and walk frames

Date: 2026-10-14, revised 2026-10-21

Tags: **[2.05]** confirmed in 2.05 rev 2 · **[srv]** GServer-v2/gs2lib only · **[cap]** captures · **[inf]** inferred.

Login line (first message, sent clean)

```
GCHAR client_type      ; 0 = PLTYPE_CLIENT           [2.05][srv]
GCHAR key              ; rand & 0x7F, gen-3 key       [2.05]
char[8] token          ; "GNW03081" for 2.05          [2.05]
GSTRING account                                       [2.05]
GSTRING password                                      [2.05]
"\n"
```

- The old "random prefix `(rand & 0x7F) + 0x20`" is the encryption key, not padding. **[2.05]**
- Each login byte is a GCHAR, so the line opens with `0x20` (type 0) and the key follows. **[2.05]**
- The token is built one character at a time in code, so there is no string literal to search for. **[2.05]**
- Right after the login line the client sends `PLI_LANGUAGE` (37, wire `0x45`) with the language string (`English` observed). **[2.05][cap]**

How the server parses it [srv]

1. Read the type GCHAR. `type = 1 << value`. 0 → client, 1 → RC, 2 → NPC-server, 3 → NC, 4 → client 2.19–2.21, 5 → client 2.22+, 6 → RC2, 8 → web.
2. For type 0, read 8 bytes as a token. 2.05 puts the key first, so these bytes are not a known token. The server rewinds one byte, switches to gen 3, reads the key, then reads the real 8-byte token.
3. Read the account and password GSTRINGs. The login line ends there for 2.05, and also for 2.22 and 2.31 (a static read of those clients' login routines finds no identity string after the password). The server's optional identity-string read is for later clients. **[2.05][bin]**
4. `GNW03081` maps to `CLVER_2_052` in current gs2lib. That enum sits **below** `CLVER_2_1`, so the server takes every pre-2.1 code path for 2.05. These include no file modtimes, `PLPROP_GANI` sent as the bow slot, the `sendLevel141` level sequence, and `gr.x`/`gr.y` movement forwarding.

Other login tokens from gs2lib: NC `NCL11012`/`NCL21075`, RC `GSERV023`–`GSERV025`, NPC-server `GRNS0000`/`LNX00001`. **[srv]**

"Walk" frames are PLI_PLAYERPROPS

The short client frames headed `"` are message id 2 (`0x22 = 2 + 0x20`), **PLI_PLAYERPROPS**. They are not a dedicated walk packet. **[srv][2.05]** The body is a property stream of `GCHAR(prop id)` + value, using the `PLPROP_*` table in `player-props.md`.

Decoding the 2026-10-04 examples **[cap][inf]**:

Frame	Props
`"1-`	`0x31-0x20 = 17` SPRITE = 13
`"/[`	15 X = 59 → 29.5 tiles
`"0f`	16 Y = 70 → 35.0 tiles
`"0[14`	Y = 59 → 29.5, then SPRITE = 20
`"*$walk/I…`	10 GANI = `walk` (length 4), then X = 41 → 20.5

- X and Y are GCHAR of `tiles * 2` (half-tile steps). SPRITE is the classic sprite index. `sprite % 4` is the direction **[srv]**.
- Direction numbering: **0 up, 1 left, 2 down, 3 right** (2.22 `commands.rtf`; GServer agrees). The "0 south / 1 west / 2 east / 3 north" mapping in the older notes is wrong.
- 2.05 builds these frames through a get-prop helper keyed by prop id. Observed groupings: {0, 11, 13, 18, 31, 33} at session start, {42, 15, 16} on attach or position, and a per-tick diff of changed props. **[2.05]**
- In 2.05 (pre-2.1), prop 10 is the **bow** slot, not GANI. The `"*$walk` frame comes from the 2.17 capture, where 10 is GANI. **[srv]**

Offline wiring

`OfflineSession::try_move` still returns `Option<PlayerUpdate>`. To put it on the wire, write `PLI_PLAYERPROPS` with X/Y/SPRITE. Drop the provisional `PacketId::Login 0x01`, `LevelRequest 0x10`, and `PlayerUpdate 0x20` ids. The real ids are type-0 login, `PLI_LEVELWARP` 0, and `PLI_PLAYERPROPS` 2.

`protocol/login-frames-by-version-2026-10-25.md`

Listserver login frames, real clients against Seihai (2026-10-25)

Derived note. Structure and sizes only. No account, password, key byte or raw login frame appears here; the account and password fields are described by length alone. Sources: lab captures of real clients against a private loopback listserver (2.05, 2.17.1, 2.22, 2.31), a lab capture of a real 2.00 (client side only), the public-listserver matrix run of 2026-10-25 22:43 JST (Seihai only, 38 runs), and Seihai `origin/main` 21a37a5 (`net/mod.rs login_listserver`, `protocol/chain.rs`).

Question

Does every real Graal client version encode its listserver login differently (beyond a version tag), so that Seihai's one classic frame, sent for every profile from 1.41 to 2.17, looks fake to the public listserver on 14922 (silent) while 14923 answers?

Short answer

No per-build difference exists in the real clients we could observe. The listserver login differs by client **generation**, in three shapes. Inside a generation every observed build sends the same bytes (2.22 and 2.31 send the same token and the same structure). Seihai's classic frame for 2.17 matches the real 2.17.1 frame in structure; what it does not match is the real 2.00/2.05 frame (id 0 only) and, apart from the two-byte zlib header, nothing else. For 2.1 to 2.16 no real client was ever observed, so no claim about "what a real one sends" can be made.

The three real shapes

Shape	Builds observed (real client)	Client to listserver	Notes
A: list name only	2.00 (lab, client side), 2.05 (lab, 14921)	ONE plain zlib frame, 9 bytes decoded (id 0 + list name `newmain` + LF), 17 bytes on the wire. Nothing else. No account, password, token or key. Client then closes.	The 2.05 receive handler reads only ids 0 and 1; its send path is the base one (no cipher).
B: id 0 then id 1	2.17.1 (lab, 14922 from signed `arrays2.dat`)	ONE plain zlib frame holding two messages: id 0 `newmain`, id 1 (length-prefixed account, length-prefixed password), LF. No token, no key, no cipher. Reply: two plain zlib frames.	Built-in host list is `arrays2.dat` (host, port 14922, fallbacks).
C: id 7 + gen-5	2.22, 2.31 (lab, 14922)	Frame 1 plain zlib, 18 bytes decoded: id 7, one key byte, token `GNW30123`, `newmain`, LF (26 on the wire). Frame 2 gen-5, comp byte 0x02, then id 1 (account, password) XOR-ciphered with the key.	The same listserver token for 2.22 and 2.31. It is not the game build token (`GNW03014`, `GNW28015`).

Per-build tokens (`GNW24011` for 2.00, `GNW03081` for 2.05, `GNW22122` for 2.17.1 ...) exist, but they go to the **game server** login, not to the listserver. Only shape C carries a token to the listserver, and it is one shared token.

Built-in listserver data from the binaries (no capture): 1.42 `<list-host-classic>:14921`, list name `newmain`; 2.00 to 2.05 the same, DES-hidden; the 2.1 test builds carry port 14922. 1.41: only a patched copy survives, original unknown.

Real versus Seihai, per profile (Seihai main 21a37a5)

Seihai picks the shape by cipher generation: Gen5 gives C, everything else gives B (id 0 plus id 1). It never sends A.

Profile	Real client login (evidence)	Seihai login	Same structure?
1.33 to 1.39	no listserver	fetches as 2.05 (B)	n/a
1.41	unknown (no binary with original host)	B	unknown
1.42	built-in host and list name known, wire frame never captured; expected A	B	probably not (extra id 1)
2.00	A (lab capture)	B	no: Seihai adds id 1 with account and password
2.01, 2.02, 2.02 rev2, 2.03, 2.04	no capture; same built-in list as 2.00/2.05, so expected A	B	probably not
2.05	A (lab capture)	B	no
2.1 to 2.16	no binary, no capture (2.1 test builds carry port 14922; wire frame never captured)	B	unknown; B is only a guess from 2.17
2.17 (2.17.1 binary)	B (lab capture)	B	yes, byte-for-byte after decompression; differs only in the zlib header (below)
2.22	C (lab capture)	C	yes (id 7 frame and the gen-5 id 1 frame decode to the same layout; sizes equal for the same account)

Profile	Real client login (evidence)	Seihai login	Same structure?
2.3	no binary; assumed C	C	assumed
2.31	C (lab capture)	C	yes

One byte-level difference that applies to every Seihai frame

The real clients compress with zlib at the best level: the stream starts `78 DA`. Seihai's zlib starts `78 9C` (default level). The deflate bodies and the checksum are identical for the same plaintext (checked on real 2.05, 2.17.1, 2.22, 2.31 frames against Seihai). The public service answers Seihai's gen-5 frames (2.22, 2.3, 2.31) with the `78 9C` header, so the header is not what makes the classic frame fail on 14922.

What the matrix shows (public service, Seihai only)

- Every classic profile (1.41, 1.42, 2.00, 2.01, 2.02, 2.02 rev2, 2.03, 2.04, 2.1, 2.12 to 2.17; 15 profiles) sent the identical frame: one plain zlib frame, 34 bytes on the wire, 26 decoded: id 0 `newmain` (9 bytes) then id 1 with a 4-byte account and a 9-byte password field (account and password masked here). Build token, version and client type do not appear. A server cannot tell these profiles apart.
- 14922: connection accepted, no byte back, no close, 5 s read timeout, for all of them.
- 14923: one plain zlib frame back after about 0.4 to 0.6 s, id 0 only, no status, site or upgrade messages, no trailing NUL bytes. Two shapes appeared with the same request: 4 rows without type prefixes (1.42, 2.00 to 2.04, 2.12, 2.14 to 2.17, 2.22) and 2 rows with `P` and `H` type prefixes (1.41, 2.02 rev2, 2.1, 2.13). The shape does not follow the version (2.1 and 2.13 got the short one, 2.12 and 2.14 the long one) and does not follow the request (identical). It looks server-side (alternating or cached views). Nothing in the reply carries a version or acceptance line.
- Gen-5 profiles (2.22, 2.3, 2.31) got the modern list on both ports; on 14923 they also got the legacy list first (4 rows plain for 2.22, a gen-5 frame for 2.3 and 2.31, and a longer list, 17 rows total, for 2.31).

Problem in the matrix data: the 2.05 rows are not a classic 2.05 login

The 2.05 runs sent shape C (id 7 with `GNW30123`, gen-5 second frame), and the public service answered it on both ports. Seihai main sends B for 2.05 (a unit test asserts it). The cause is in `bin/main.rs`: when the parsed version equals the default (2.05), the saved GUI version from the settings file replaces it, even when `--version 2.05` was given on the command line. The user's GUI had a gen-5 version saved, so every `--version 2.05` run ran as that profile. Consequences:

1. The matrix and the first user run have no real 2.05 data point. "2.05 got the modern list on 14922" is the saved profile's result.
2. The port rule "2.05 and later use 14922" rests on gen-5 logins only (2.22, 2.3, 2.31 and the masked 2.05 runs). A classic 2.05 (B) was never tried on the public service. Likely result on 14922: silent, like the other classic profiles.
3. The fix is one line: apply the saved version only when `--version` was not given (`version_given` is already tracked). Not done in this note's work.

Does the evidence support "classic logins look fake"?

- For a version tag: there is none in classic frames at all, in the real clients as well as in Seihai. The real 2.17.1 frame has none either.
- For a per-version encoding: none exists inside a generation. 2.22 and 2.31 are identical on the wire; 2.00 and 2.05 are identical; the classic frame has no field that could carry a build.

- What the data does show: the public service answers gen-5 logins (shape C) on 14922 and ignores the 2.17-style frame (shape B) there. A real 2.17.1 sends shape B to 14922 (its signed `arrays2.dat` says so). Whether the public service answers a real 2.17.1 on 14922 was never observed; if it also ignores it, the real client falls back to the other hosts in `arrays2.dat`, or the service simply does not support classic logins on 14922 (consistent with 14923 being the legacy port).
- The only detectable differences between Seihai's B frame and the real 2.17.1 B frame are the two zlib header bytes (`9C` against `DA` as second byte, with the matching header check bits). The gen-5 evidence says the header is not checked, but it was not tested on a classic frame.

What Seihai would change to look like the real clients

1. 2.00 to 2.05 (and 1.42, 2.01 to 2.04 as a best guess): send shape A, id 0 plus `newmain` only. This is a real, captured difference. Port 14923 would still be expected to answer (the legacy port answered B, and A is what the real clients send there).
2. All shapes: compress with the best zlib level (header `78 DA`) so the stream matches the real clients byte for byte. Cheap and risk-free.
3. 2.1 to 2.16: nothing to match against. Keep B (the 2.17 shape) or A and keep it marked inferred.
4. Fix the saved-version override so `--version 2.05` really runs 2.05.

Confidence

- High: the three shapes; that 2.22 and 2.31 are identical to Seihai in structure; that 2.00/2.05 send id 0 only; that all 15 classic Seihai profiles send one identical frame; the zlib header difference; the matrix replies.
- Medium-high: the 2.05 override explanation (read from code and consistent with the gen-5 frames in the 2.05 runs; the user's saved version has not been inspected).
- Low: why 14922 stays silent for B (never tested with a real 2.17.1 or with a `78 DA` classic frame); whether 1.41, 1.42, 2.01 to 2.04 send A or something else; what 2.1 to 2.16 send.

Captures that would settle it (all read-only list requests)

1. A real 2.17.1 client (`Graal_2171.exe`, pristine, its signed `arrays2.dat`) run on a machine that can reach the service, with a packet capture, started and closed at the list. Settles whether a real classic login also gets nothing on 14922, and what a real client does next (falls back, retries, or shows a list).
2. A real 2.00 or 2.05 client with its built-in host redirected to the public listserver on 14923 (hosts file), capturing the list exchange. Settles shape A on the real service.
3. Seihai, after the override fix: one list request each for the classic 2.05 profile on 14922 and 14923, plus a build with `78 DA` compression for 2.17 on 14922. If `78 DA` makes 14922 answer, the header is the tell; if not, 14922 simply does not serve classic logins.

`protocol/login-recovery-start.md`

Protocol recovery — initial notes

Date: 2026-10-14

Starting point

Newest binary: Graal_231.exe (post IDR + Dhrake).

Observed in EntryPoint (231)

- Auto-connect path when two controls (likely username + password) have non-empty text.
- Calls a handler equivalent to `TConnectFrame_Button1Click`.
- This is the highest-value next trace target for the login packet.

Immediate next trace

1. Locate the connect button handler in the 231 Ghidra database.
2. Follow the call chain that builds and sends the initial login / handshake packet.
3. Record:
    - Packet ID / first byte(s)
    - Field order and encoding (length-prefixed strings, etc.)
    - Any version or client-identification bytes
4. Add the corresponding Rust type + serializer in `seihai/client/src/protocol`.

Provisional implementation (2026-10-14)

A working (but unconfirmed) encoding is now in `seihai/client`: - `LoginRequest` — id 0x01, length-prefixed username/password/version - `LevelRequest` — id 0x10, length-prefixed name - `PlayerUpdate` — id 0x20, f32 x/y + u8 dir

These exist so the client can attempt a connection. Replace the layouts as soon as the real packet formats are recovered from the 231 binary.

Other high-value flows (later)

- Level request / level data
- Player movement / position updates
- NPC / script-related messages

`protocol/login-session-known.md`

What a 5.x/6.x Login server is already known to send

Date: 2026-11-01. No socket was opened. This collects the protocol notes and the client reads that already describe a server the client names `Login`. It leaves matrix unknown #3 open: the screens a public Login server paints are still unobserved.

Tags follow the notes they come from. A lab line is a loopback GServer-v2 standing in under that name. It is a record of what these clients accept.

Handlers the client ties to that session

The 5.x and 6.01/6.03 clients open an ordinary game connection and name the server `Login`. There is no separate list protocol. The server list and the account screens are scripted GUI from that session. **[bin]** `protocol/v5x-clients.md`, `protocol/v6x-clients.md`.

Message the notes tie to this session	What the client does with it	Tag
S→C 190 `PLO_SERVERLISTCONNECTED`	Raises `onServerListerConnect`. 4.20r8 treats this as the Login server accepting the session. 5.00r7 ties it to the server-list screen. `packet-ids.md` leaves the payload cell empty.	**[bin]**
S→C 178 `PLO_SERVERWARP`	Comma text of at least four fields. Field 0 is unused, field 1 is the new server name, field 2 the host, field 3 the port. The client closes the session and logs in again under that name. Seen when a lab proxy injected it, and when GServer-v2's listserver forwards a server-info request.	**[bin]** `packet-ids.md`; **[live]** `client-internals/live-behaviour-404.md`; **[cap]** `version-profiles/lab-v5-2026-10-30.md`, `protocol/v5v6-lab-play.md`
Level-load ids (name, board, chests, NPCs), on the lab stand-in	While the server name starts with `login`, the draw-layer mask is 0. 4.04r2 draws no players and no HUD. 5.x/6.x draw no characters and no default HUD. 5.24 and 5.30 draw tiles; 5.31r4 and 6.03r7 draw chests and image NPCs and no tiles until after the warp.	**[bin]** 4.04 mask; **[cap]** `lab-v5-2026-10-30.md`, `v5v6-lab-play.md`
Script and gani handlers (S→C 140, 197, 195)	The same handlers a game server uses. The notes call the server list, the account screens and the chat bar scripted GUI. The engine looks up a GUI control named `ChatBar` before keyboard input becomes chat. A bare lab server creates no such control, and typing sends nothing. The script text itself is in the unobserved list below.	**[bin]** `v5x-clients.md`, `v4x-clients.md`; **[cap]** `lab-v5-2026-10-30.md`
File reply to the client's `basepackage.gupd` request (C→S 23, or 47 when that server's copy is cached)	File ids carry the body (100/102), 45 answers "not modified", 105/106 mark package size and completion. A minimal `GRPKG001` with `NAME`, `VERSION`, `PLATFORM` and `MODE` satisfies the lab client. The client stores the Login copy under `updatepackages/`.	**[bin]** `v4x-clients.md`; **[cap]** `file-transfer.md`, `v5v6-lab-play.md`, `live-behaviour-404.md`

6.01 and 6.03 use the same named session and the same 190 event. Their extra receive ids (109, 111 `creationtime.dat`, 198) are general handlers. No note says a Login server is the one that sends them. **[bin]** `v6x-clients.md`.

6.11 asks for a connector script before the game socket. The reply is loaded as `StartScript_Connector` and its `onCreated` runs. That script calls `connecttogameserver` and can replace the framing, the cipher and both id tables, and it builds the login itself. The gen-5 path remains the default until the script calls `setprotocol`. **[bin]** `v6x-clients.md`.

The lab burst these clients accepted

`v5v6-lab-play.md` records GServer-v2's first burst after the login frame, on every 5.x/6.x build that lab played. The server side is GServer-v2. Ids in that burst include 190, then the level load (14, 6, 100, 101, chests, 156, 3), the staff and status lists (47, 180), the start and HTML messages (179, 41) and 182 (the client answers with its window titles on 44). 178 is absent from that burst. The lab proxy sends 178 afterwards. GServer-v2 itself sends 178 when its listserver forwards a server-info request. **[wire]**

4.04r2 on the same kind of stand-in asks for `basepackage.gupd` and `bigmap.txt` on the Login session. `bigmap.txt` is asked once per run of the program, on that session, and is not asked again after the warp. **[live]** `live-behaviour-404.md`.

Still unobserved

A public Login session was not captured.

- The bytes a public 5.x or 6.x Login server sends, including the payload of S→C 190.
- The GUI script text for the server list, the account screens and the `ChatBar` control.
- The `FILE` list inside that server's `basepackage.gupd`, and any later update package.
- The backdrop level that server chooses (name, tiles, chests, image NPCs).
- The 6.11 connector script body, and with it the framing, cipher and login token a 6.11 client actually uses.
- The resulting look (matrix unknown #3). The lab screens are the lab world's ganis and a server named by the injected 178.

`protocol/medium-term-expansion.md`

Medium-term expansion

Date: 2026-10-14

Renderer

- `ascii_board` — full board, direction character for the player (`v < > ^`)
- `ascii_camera` — window centered on the player, out-of-bounds as space
- `write_ppm` — 64×64 PPM, walls dark, water blue, player red
- Tile characters: # wall, ~ water, `.` passable

SDL2 remains unavailable. These are the offline view until a graphics dependency is added.

Listserver / net

- `Connection::listserver_session` — connect, classic login, read reply
- `Connection::send_walk` — short walk frame
- `parse_server_list` — provisional line filter (skips `GNW*` lines)

No decryption. List framing is still provisional.

GS1

- Arithmetic, comparison, jump opcodes execute
- `Commands` registry with offline stubs: `message`/`say`, `setimg`, `move`
- `CmdCall` dispatches into the registry
- `Bytecode::execute_with` accepts a custom registry

`Call`, array ops, and most GS1 commands remain no-ops.

Separation

Raw decompiles stay in `/shdecompiling/`. This note is cleaned. Client code is in `/seihai/`.

`protocol/medium-term-slice.md`

Medium-term slice

Date: 2026-10-14

Renderer

SDL2 is not available in this environment. `client/src/render/mod.rs` provides: - `ascii_board` — 64×64 ASCII view (`#` wall, `.` passable, `P` player) - `write_ppm` — raw PPM dump of the same view

A real SDL2/OpenGL path can replace this when the dependency is present.

Listserver

`Connection::listserver_session` connects, sends the locked classic login line, and reads the reply as text. No decryption. Timeout is 5 seconds.

GS1 interpreter

`Bytecode::execute` runs a minimal stack VM covering the OpenGS1 arithmetic, comparison, and jump opcodes. `Call` / `CmdCall` / array ops are no-ops until the command set is wired. Unit test covers `Push` + `Add`.

`protocol/npc-control.md`

NPC-Control protocol

Date: 2026-10-31

Wire layouts for the NPC-server (NC) socket that staff tools use. Written for `seihai-rc` stage 5. Sources: static reads already filed as `tools/npc-control.md` (RC 1.41 / 2002) and `tools/remotecontrol.md` (2004 Delphi `RemoteControl.exe`); Drive inventory of Squirrel's `NC [NPC Control]` and `RC [Remote Control]` folders (binaries and docs held outside the repo). Tags: **[bin]** static read · **[doc]** 2001 NPCControl text · **[srv]** gs2lib / GServer names · **[inf]** inferred, needs a loopback check.

Raw decompiles and client binaries stay on Drive. This note has plain behaviour only.

Login

Socket	First message	Framing after login
NC	type **3**, token `NCL11012`, `str account`, `str password` (no key byte)	fixed zlib frame (`u16be` length + zlib body), **no** type byte, **no** cipher, for the whole session **[bin]**

- Message body: classic Graal text lines (`GCHAR id` + payload + `\n`), G-integers with `+0x20`. Batches flush at 0x1000 bytes. **[bin]**
- Large uploads: id 50 (`PLI_RAWDATA`) with a GINT3 length, then one raw frame; receive id 100 (`PLO_RAWDATA`) switches the reader for the next *length* bytes. **[bin]**
- The NC login did not change between RC 1.41 (2002) and the 2004 RemoteControl. The RC game-server login did (`GSERV023` type 1 → `GSERV024` type 6 / gen 5). **[bin]**

- Default NC host fields in the 2004 binary are the loopback address and `14804` (lab ports; unused leftovers once discovery works). **[bin]**

Finding the NPC server [bin]

The RC does not ask the user for an NC address. It:

1. Treats any player whose nickname contains `(Server)` as an NPC-server player.
2. Sends id **94** (`PLI_NPCSERVERQUERY`): `GSHORT player id` + text `location`.
3. Handles id **79** (`PLO_NPCSERVERADDR`): `GSHORT player id`, then `ip,port`. Opens the NC socket only if that nickname still contains `(Server)` and NC is not already connected.
4. Sends the NC login above. Status: "Connected to NPC-Server".

The 2001 doc (`npccontrol.txt` in the NPCControl 1.3 bundle) says the NewWorld game server exposed NC on port **14950**, with NewWorld accounts allowed and main-server accounts needing admin level ≥ 4. **[doc]** That is the older published port; live discovery uses packet 79.

NC → client (server ids)

Handled on the NC socket (RC 1.41). Everything else on that socket, including 16, still goes through the normal RC handler. **[bin]**

Id	gs2lib name	Layout	UI / effect
16	DISCMESSAGE	rest = text	disconnect box
33	NPCWEAPONADD	`str name`, `str image`, `GSHORT len`, `len` bytes of script	weapon script editor
80	NC_LEVELLIST	rest = level list, one per line	level list editor
157	NC_NPCATTRIBUTES	rest = attribute text, one per line	attribute viewer (view-only)
158	NC_NPCADD	`GINT3 id`, then props: `GCHAR 50` + `str name`, `GCHAR 51` + `str type`, `GCHAR 52` + `str level` (any order, any subset)	add/update NPC list row
159	NC_NPCDELETE	`GINT3 id`	remove row
160	NC_NPCSCRIPT	`GINT3 id`, rest = script	script editor
161	NC_NPCFLAGS	`GINT3 id`, rest = flags, one per line	flag editor
162	NC_CLASSGET	`str name`, rest = script	class editor
163	NC_CLASSADD	rest = class name	add to class list
164	NC_LEVELDUMP	rest = text	dump viewer (view-only)
167	NC_WEAPONLISTGET	repeated `str name`	weapon list
188	NC_CLASSDELETE	class name **[inf]**	handled in the 2004 build; not in 2002 **[bin]**

2004 RemoteControl also lists NC receive ids 16, 33, 80, 157–164, 167, 188. **[bin]**

Client → NC (client ids)

Id	gs2lib name	From	Payload
3	(login)	connect	see Login
79	RC_CHAT	console line typed in NC mode	text

Id	gs2lib name	From	Payload
103	NC_NPCGET	NPC list: open / attributes	`GINT3 id`
104	NC_NPCDELETE	NPC list: Delete (confirm)	`GINT3 id`
105	NC_NPCRESET	NPC list: Reset	`GINT3 id`
106	NC_NPCSCRIPTGET	NPC list: Script	`GINT3 id`
107	NC_NPCWARP	Quick warp / warp dialog	`GINT3 id`, `GCHAR x*2`, `GCHAR y*2`, level
108	NC_NPCFLAGSGET	Edit flags	`GINT3 id`
109	NC_NPCSCRIPTSET	script editor: Save	`GINT3 id`, script
110	NC_NPCFLAGSSET	flag editor: Save	`GINT3 id`, flags
111	NC_NPCADD	Add NPC dialog	comma text: name, id, type, scripter, start level, x, y. Type drop-down: `OBJECT`, `ANIMAL`, `POLICE`, `HORSE`, `BOAT`, `HOUSEC`
112	NC_CLASSEDIT	class list: open	class name
113	NC_CLASSADD	class editor: Save	`str name`, rest = script
114	NC_LOCALNPCSGET	Local NPCs dialog	level name
115	NC_WEAPONLISTGET	console button	—
116	NC_WEAPONGET	weapon list: open	weapon name
117	NC_WEAPONADD	weapon editor: Save	`str name`, `str image`, rest = script
118	NC_WEAPONDELETE	weapon list: Delete	weapon name
119	NC_CLASSDELETE	class delete	class name — **sent in 2004**; no button in 2002 **[bin]**
150	NC_LEVELLISTGET	console button	—
151	NC_LEVELLISTSET	level list editor: Save	level list

2004 also sends NC ids 3, 79, 103–119, 150, 151. **[bin]**

Shared editor window modes map Save to 109 / 110 / 113 / 117 / 151. Attributes (157) and dumps (164) are view-only (can save as `.rtf` locally). **[bin]**

Id 111 matches the 2001 PM `add npc` field list moved onto its own packet. **[inf][doc]**

Cross-links

- RC (game-server) staff packets and gen-5 framing: `tools/remotecontrol.md`, `tools/remotecontrol-lineage.md`.
- 2001 NPC-server PM admin channel (older than this socket): `tools/npcserver-2001.md`.
- Packet id names: `protocol/packet-ids.md`. Staff overview: `protocol/chat-pm-rc.md`.

Drive material (inventory only; not in git)

Item	Drive id	Role
Folder `NC [NPC Control]`	`1sABQxJiJ4cwbtfzrB9oqCwNhC5G3Zqsz`	NPCControl1.3 / 13 rars, `nc7132001`, NPC Server Document
Folder `RC [Remote Control]`	`1oF6qDwxTMoIqmHIugbx_O72m3B7_Qjih`	RC rars (rc139, rc2, Custom RemoteControl, windows_rc3_beta, …)
`RemoteControl.exe`	`1WAoHpifrHvM72aYXVNzkKRJVTkgJNkGH`	2004 Delphi staff client (~1 MB)
`RemoteControl.exe.c` / `.html`	under `Graal Decompiles` / Accessories	decompile; read locally, never commit
`RC2_2.05.exe` (+ `.c`)	`1J7Ac-F42R1t8siRcveYO8Np2Il0ebbAP`	GTK RC2 line

Check on loopback [inf]

Confirm against GServer-v2 lab + Wine `RemoteControl.exe` / RC 1.41 (sandbox, the loopback address, lab ports only):

1. Exact bytes of NC login (type 3 / `NCL11012` / account / password) and whether any junk byte follows.
2. Packet 79 address string format (`ip,port` vs spaces) when the lab NPC-server player appears.
3. NC_NPCADD (158) property order when the server sends all three props; whether missing props clear or leave the row.
4. NC_CLASSDELETE (188 / client 119): payload is bare class name vs length-prefixed string.
5. Weapon add (33) script length: `GSHORT` vs GINT3 on large scripts; RAWDATA path for big set packets (109 / 113 / 117).
6. Local-NPC list reply id and layout after client 114 (not named in the 2002 handler table above).
7. Whether id 16 on the NC socket really shares the RC disconnect UI when the NPC-server drops the link.

`protocol/offline-expansion.md`

Offline expansion

Date: 2026-10-14

Offline session

- `in_water` tracks type-11 tiles. Does not block.
- `walk_gani` is an optional `GaniPlayer`. Stepped on every successful move.
- `set_walk_gani` / `walk_frame` expose it.

GS1 command stubs

Registered no-ops / pass-throughs: - `message`, `say` — return the first argument - `setimg`, `showimg` — return the first argument - `move`, `hideimg`, `setlevel`, `warp` — return None

`CmdCall` dispatches into the registry. Movement and level changes stay with the session, not the VM.

Scope

Offline only. No listserver, no new decompile work.

`protocol/OPCODE-SUMMARY.md`

Opcode Summary — 39ster 2.1

Authoritative source: `msgFuncs` table + enums in original `player.h`.
Full details and emission points: `ash/OPCODE-MAP.md`.

Client → Server (selected)

Op	Name	Notes
0	LEVELWARP	

Op	Name	Notes
1	BOARDMODIFY	
2	PLAYERPROPS	
3	NPCPROPS	
4	ADDBOMB	
20	OPENCHEST	remembers coords
23	WANTGIF	file request
30	MODLEVELWARP	
37	LANGUAGE	
51–87	RC ops	mostly empty stubs

Server → Client (selected)

Op	Name	Notes
1	LEVELLINK	
2	SBADDYPROPS	
3	SNPCPROPS	
6	LEVELNAME	
7	SBOARDMODIFY	
8	OTHERPLPROPS	
9	SPLAYERPROPS	
100	FILELEN	
101	TILES	GR-V1 board
102	FILEBLOB	

See Ash OPCODE-MAP.md for the complete 0–88 client table and all server ops.

Note (2026-10-21)

This table follows 39ster 2.1. Current gs2lib names: 23 = `PLI_WANTFILE`; 30 = `PLI_LEVELWARPMOD`; server 100/101/102 = `PLO_RAWDATA`/`PLO_BOARDPACKET`/`PLO_FILE`, with the same roles as FILELEN/TILES/FILEBLOB. 2.05 coverage and the full id list are in `packet-ids.md`.

`protocol/packet-id-diff.md`

Packet-id cross-check — 1.39 vs v2.05

Date: 2026-10-14

Sources

- 1.39: `shdecompiling/graal-139-dhrake/Graal 139.exe-2.c` (Dhrake)
- v2.05: `shdecompiling/graal-v205/DispatchGs1.txt` and `Graal.exe.c`

Finding

`DispatchGs1.txt` is the GS1 interpreter, not the network packet dispatcher. It switches on script opcodes, not wire packet ids. A direct case-by-case packet-id diff against it is not meaningful.

What the captures lock

Logical	Wire	Role	Confidence
0x02	" (0x22)	walk / player update	high — clear sessions, research notes
—	(no type byte)	login line	high — prefix + GNW##### + length-biased fields
—	server→client	board modify	medium — PLO_BOARDMODIFY in GServer-v2

Case-pattern notes

- 1.39 export uses `case 0xNN:` (586 hex cases). Includes `case 0x10:` in several switches; context is mixed (not all packet dispatch).
- v2.05 export uses `case (code)0xN:` inside `DispatchGs1` (script opcodes).

Provisional ids still in Seihai

`PacketId::Login` (0x01), `LevelRequest` (0x10), `PlayerUpdate` (0x20) remain provisional. The classic login line and the walk frame do not use these ids; they use the shapes above.

Next

Locate the actual receive switch in the 1.39 export (search for the session read loop, not `DispatchGs1`) and name the arms against the capture shapes.

Update 2026-10-21 (resolved)

- The 2.05 network receive switch is now located. It is a jump table over `id = byte − 0x20` with bound 0xA8. Its arms match gs2lib `PLO_*` names for 0–25, 27–48, 75, 100, 150, 151, 153–156, 165, 166, and 168. 26 (`PLO_NPCACTION`) is not handled. **[2.05]**
- Client → server senders exist for ids 1–38 (except 6, 25, and 31), plus 80 and 81. Id 0 covers both the login line and `PLI_LEVELWARP`. **[2.05]**
- The provisional Seihai ids are retired. Login is client type 0, level request is `PLI_LEVELWARP` (0) or `PLI_ADJACENTLEVEL` (35), and the player update is `PLI_PLAYERPROPS` (2).
- The full table is in `packet-ids.md`. The 1.39 receive switch has not been redone yet. Expect the same ids with fewer arms.

`protocol/packet-ids.md`

Packet id tables

Date: 2026-10-21

Names: gs2lib `IEnums.h` (2026-10 bitbucket head, GPLv3). The names are as in gs2lib; the notes are restated in this repo's field notation (`c`, `s`, `i3`, `str`, …), not copied. The 2.05 column is taken from the 2.05 rev 2 server→client jump table and the client→server sender functions.

Wire byte = id + 0x20 (GCHAR). Ids above 223 cannot be sent as a single GCHAR.

Legend, 2.05 column: ✔ handled or sent by 2.05 · ・ not present in 2.05.

Server → client (PLO_*)

Id	Wire	Name	2.05	Notes (restated from server source)
0	0x20	`PLO_LEVELBOARD`	✓	
1	0x21	`PLO_LEVELLINK`	✓	
2	0x22	`PLO_BADDYPROPS`	✓	
3	0x23	`PLO_NPCPROPS`	✓	
4	0x24	`PLO_LEVELCHEST`	✓	
5	0x25	`PLO_LEVELSIGN`	✓	
6	0x26	`PLO_LEVELNAME`	✓	
7	0x27	`PLO_BOARDMODIFY`	✓	
8	0x28	`PLO_OTHERPLPROPS`	✓	
9	0x29	`PLO_PLAYERPROPS`	✓	
10	0x2a	`PLO_ISLEADER`	✓	
11	0x2b	`PLO_BOMBADD`	✓	
12	0x2c	`PLO_BOMBDEL`	✓	
13	0x2d	`PLO_TOALL`	✓	
14	0x2e	`PLO_PLAYERWARP`	✓	
15	0x2f	`PLO_WARPFAILED`	✓	
16	0x30	`PLO_DISCMESSAGE`	✓	
17	0x31	`PLO_HORSEADD`	✓	
18	0x32	`PLO_HORSEDEL`	✓	
19	0x33	`PLO_ARROWADD`	✓	
20	0x34	`PLO_FIRESPY`	✓	
21	0x35	`PLO_THROWCARRIED`	✓	
22	0x36	`PLO_ITEMADD`	✓	
23	0x37	`PLO_ITEMDEL`	✓	
24	0x38	`PLO_NPCMOVED`	✓	
25	0x39	`PLO_SIGNATURE`	✓	`c v`: 1 = the client quits; 0 = quits unless the host is the loopback address; other values ignored (layout read in 2.17–2.31; the id is handled from 2.00) **[bin]**
26	0x3a	`PLO_NPCACTION`	·	purpose unknown; client 6.037 ignores it
27	0x3b	`PLO_BADDYHURT`	✓	
28	0x3c	`PLO_FLAGSET`	✓	
29	0x3d	`PLO_NPCDEL`	✓	
30	0x3e	`PLO_FILESENDFAILED`	✓	
31	0x3f	`PLO_FLAGDEL`	✓	
32	0x40	`PLO_SHOWIMGPLAYER`	✓	`GSHORT sender id` + the sender's 24 payload, relayed unchanged by the stock server to the others in the level (lab) **[cap]**
33	0x41	`PLO_NPCWEAPONADD`	✓	
34	0x42	`PLO_NPCWEAPONDEL`	✓	
35	0x43	`PLO_RC_ADMINMESSAGE`	✓	

Id	Wire	Name	2.05	Notes (restated from server source)
36	0x44	`PLO_EXPLOSION`	✓	
37	0x45	`PLO_PRIVATEMESSAGE`	✓	
38	0x46	`PLO_PUSHAWAY`	✓	`c dx`, `c dy`; each scales as v/16 − 4 (±2 tiles in 1/16 steps)
39	0x47	`PLO_LEVELMODTIME`	✓	2.17–2.31: the rest is passed unparsed to the active gmap or level object, only if one exists **[bin]**
40	0x48	`PLO_HURTPLAYER`	✓	
41	0x49	`PLO_STARTMESSAGE`	✓	client 6.037 ignores it; rest = RTF or HTML text; cached as `servermessage.rtf` / `.html` and shown in the start-message viewer; client 6.037 ignores it **[bin]**
42	0x4a	`PLO_NEWWORLDTIME`	✓	
43	0x4b	`PLO_DEFAULTWEAPON`	✓	
44	0x4c	`PLO_HASNPCSERVER`	✓	when present, the client stops applying NPC props itself; client 5.07+ ignores it; no payload; handled by 2.00–2.22, not handled by 2.31 **[bin]**
45	0x4d	`PLO_FILEUPTODATE`	✓	
46	0x4e	`PLO_HITOBJECTS`	✓	
47	0x4f	`PLO_STAFFGUILDS`	✓	comma text, stored as the staff-guild list **[bin]**
48	0x50	`PLO_TRIGGERACTION`	✓	`s fromPlayer`, `i3 fromNPC`, `c xTile`, `c yTile`, `list action,params`
49	0x51	`PLO_PLAYERWARP2`	·	`c x×2`, `c y×2`, `c z` (unused by 2.17–2.31), `c gmapX`, `c gmapY`, rest = level; x and y are `v/2` tiles; gmapX/Y apply only when the level name ends in `.gmap` (clamped to the grid) **[bin]**
50	0x52	`PLO_RC_ACCOUNTADD`	·	
51	0x53	`PLO_RC_ACCOUNTSTATUS`	·	
52	0x54	`PLO_RC_ACCOUNTNAME`	·	
53	0x55	`PLO_RC_ACCOUNTDEL`	·	
54	0x56	`PLO_RC_ACCOUNTPROPS`	·	
55	0x57	`PLO_ADDPLAYER`	·	client 5.07+ ignores it
56	0x58	`PLO_DELPLAYER`	·	client 5.07+ ignores it
57	0x59	`PLO_RC_ACCOUNTPROPSGET`	·	

Id	Wire	Name	2.05	Notes (restated from server source)
58	0x5a	`PLO_RC_ACCOUNTCHANGE`	·	
59	0x5b	`PLO_RC_PLAYERPROPSCHANGE`	·	
60	0x5c	`PLO_UNKNOWN60`	·	client 5.07+ ignores it
61	0x5d	`PLO_RC_SERVERFLAGSGET`	·	
62	0x5e	`PLO_RC_PLAYERRIGHTSGET`	·	
63	0x5f	`PLO_RC_PLAYERCOMMENTSGET`	·	
64	0x60	`PLO_RC_PLAYERBANGET`	·	
65	0x61	`PLO_RC_FILEBROWSER_DIRLIST`	·	
66	0x62	`PLO_RC_FILEBROWSER_DIR`	·	
67	0x63	`PLO_RC_FILEBROWSER_MESSAGE`	·	
68	0x64	`PLO_LARGEFILESTART`	·	rest = file name (2.31 also clears the stored size) **[bin]**
69	0x65	`PLO_LARGEFILEEND`	·	rest = file name; if it equals the stored name the received stream is processed as a file **[bin]**
70	0x66	`PLO_RC_ACCOUNTLISTGET`	·	
71	0x67	`PLO_RC_PLAYERPROPS`	·	
72	0x68	`PLO_RC_PLAYERPROPSGET`	·	
73	0x69	`PLO_RC_ACCOUNTGET`	·	
74	0x6a	`PLO_RC_CHAT`	·	
75	0x6b	`PLO_PROFILE`	✓	repeat `c n` + n chars; first field = account, the rest = profile lines **[bin]**
76	0x6c	`PLO_RC_SERVEROPTIONSGET`	·	
77	0x6d	`PLO_RC_FOLDERCONFIGGET`	·	
78	0x6e	`PLO_NC_CONTROL`	·	repurposed by Graal Reborn; original purpose unknown; client 5.07+ ignores it
79	0x6f	`PLO_NPCSERVERADDR`	·	two bytes (0, 2), then `ip,port` text; client 5. ignores it
80	0x70	`PLO_NC_LEVELLIST`	·	`str levels`; client 5.07+ ignores it
81	0x71	`PLO_UNKNOWN81`	·	client 5.07+ ignores it
82	0x72	`PLO_SERVERTEXT`	·	reply to PLI_REQUESTTEXT / PLI_SENDTEXT
83	0x73	`PLO_UNKNOWN83`	·	client 5.07+ ignores it

Id	Wire	Name	2.05	Notes (restated from server source)
84	0x74	PLO_LARGEFILESIZE	·	`i5 size` (2.31 only) **[bin]**
100	0x84	PLO_RAWDATA	✓	`i3 length`
101	0x85	PLO_BOARDPACKET	✓ via RAWDATA	
102	0x86	PLO_FILE	✓ via RAWDATA	
103	0x87	PLO_RC_MAXUPLOADFILESIZE	·	`i5 bytes`: RC upload limit (default 20 MiB)
104	0x88	PLO_UNKNOWN104	·	`str data` holding `c`, `i5`, `i5` (an 8-byte value)
105	0x89	PLO_UPDATEPACKAGESIZE	·	package-update message (5.07, 6.037)
106	0x8a	PLO_UPDATEPACKAGEDONE	·	package-update message (5.07, 6.037)
107	0x8b	PLO_BOARDLAYER	·	like LEVELBOARD, for extra layers
108	0x8c	PLO_UNKNOWN108	·	
109	0x8d	PLO_UNKNOWN109	·	`i5 modtime`, `str url`: fetch a file over HTTP (connector?)
110	0x8e	PLO_UNKNOWN110	·	ignored by clients
111	0x8f	PLO_SETNETCOOKIE	·	`rest cookie`
124	0x9c	PLO_UNKNOWN124	·	seen by RC3; possibly player flags (untested)
131	0xa3	PLO_NPCBYTECODE	·	`i3 code`, `bytecode`
132	0xa4	PLO_UNKNOWN132	·	5.07 only; client 6.037 ignores it
133	0xa5	PLO_UNKNOWN133	·	rest = text; maybe registers a class script
134	0xa6	PLO_GANISCRIPT	·	gani scripts, maybe for package downloads
140	0xac	PLO_NPCWEAPONSCRIPT	·	`s info_length`, `script`
150	0xb6	PLO_NPCDEL2	✓	`c n`, n-char level name, `i3 npc id` **[bin]**
151	0xb7	PLO_HIDENPCS	✓	no payload; removes NPCs with ids 1 to 999 of the active level (id 0 excluded) **[bin]**
153	0xb9	PLO_SAY2	✓	rest = text; signs use it too
154	0xba	PLO_FREEZEPLAYER2	✓	
155	0xbb	PLO_UNFREEZEPLAYER	✓	
156	0xbc	PLO_SETACTIVELEVEL	✓	rest = level or `.gmap` name; a `.gmap` is loaded and made active. 2.31 adds an extra follow-up step (purpose unknown) **[bin]**
157	0xbd	PLO_NC_NPCATTRIBUTES	·	`str attributes`; client 5.07+ ignores it
158	0xbe	PLO_NC_NPCADD	·	`i3 id`, `c 50`, `str name`, `c 51`, `str type`, `c 52`, `str level`

Id	Wire	Name	2.05	Notes (restated from server source)
159	0xbf	`PLO_NC_NPCDELETE`	·	`i3 id`; client 5.07+ ignores it
160	0xc0	`PLO_NC_NPCSCRIPT`	·	`i3 id`, `str script`; client 5.07+ ignores it
161	0xc1	`PLO_NC_NPCFLAGS`	·	`i3 id`, `str flags`; client 5.07+ ignores it
162	0xc2	`PLO_NC_CLASSGET`	·	`str name`, `str script`; client 5.07+ ignores it
163	0xc3	`PLO_NC_CLASSADD`	·	`class`; client 5.07+ ignores it
164	0xc4	`PLO_NC_LEVELDUMP`	·	client 5.07+ ignores it
165	0xc5	`PLO_MOVE`	✓	client 6.037 ignores it; `i3 npc id` (> 0) + move data **[bin]**
166	0xc6	`PLO_SHOWIMGNPC`	✓	`i3 npc id` + prop list **[bin]**
167	0xc7	`PLO_NC_WEAPONLISTGET`	·	`c name1 length`, `name1`, `c name2 length`, `name2`; client 5.07+ ignores it
168	0xc8	`PLO_UNKNOWN168`	✓	sent by the login server; payload ignored; local-player reset (2.17–2.31) **[bin]**
169	0xc9	`PLO_UPGRADEURL`	·	with DISCONNECT: the reconnect button becomes "Upgrade" and opens this URL; rest = URL text **[bin]**
170	0xca	`PLO_GHOSTMODE`	·	`c flag` (> 0 = ghost on) **[bin]**
171	0xcb	`PLO_BIGMAP`	·	client 6.037 ignores it; comma text `txt,png,x,y` (≥ 4 fields used); x, y parsed as floats **[bin]**
172	0xcc	`PLO_MINIMAP`	·	same layout as 171, goes to the minimap **[bin]**
173	0xcd	`PLO_GHOSTTEXT`	·	rest = text **[bin]**
174	0xce	`PLO_GHOSTICON`	·	`c flag` (> 0 = show icon) **[bin]**
175	0xcf	`PLO_SHOOT`	·	client 6.037 ignores it; `s player id` + shot data; only when the player is in the active level **[bin]**
176	0xd0	`PLO_KINGDOMSMODE`	·	client switches to Kingdoms mode; no payload **[bin]**
177	0xd1	`PLO_NEWWORLDMODE`	·	client switches to New World mode; no payload **[bin]**

Id	Wire	Name	2.05	Notes (restated from server source)
178	0xd2	`PLO_SERVERWARP`	·	comma text, ≥ 4 fields: field 0 unused, 1 = server name, 2 = host, 3 = port; the client closes the connection and reconnects with a fresh login, no listserver step (2.31 lab; the lab text came from the open-source listserver reimplementation) **[bin] [cap]**
179	0xd3	`PLO_RPGWINDOW`	·	rest = text **[bin]**
180	0xd4	`PLO_STATUSLIST`	·	rest = text **[bin]**
181	0xd5	`PLO_UNKNOWN181`	·	`s player id`, `c toggle` (length ≥ 3) **[bin]**
182	0xd6	`PLO_LISTPROCESSES`	·	no payload; client answers `PLI_PROCESSLIST` (44) **[bin]**
183	0xd7	`PLO_HASPROCESSRUNNING`	·	rest = names; client answers `PLI_PROCESSRUNNING` (45) with `c 0/1` **[bin]**
184	0xd8	`PLO_TAKESCREENSHOT`	·	rest = file name, only `.png` accepted; saved under a screenshots folder **[bin]**
185	0xd9	`PLO_BOARDHEIGHTS`	·	`c mapX`, `c mapY`, `c blockX`, `c blockY`, `c w-1`, `c h-1`, then w×h cells of 2 raw bytes; height = (b0−0x20−50) + (b1−0x20)/128 **[bin]**
186	0xda	`PLO_BOARDMODIFY2`	·	`c mapX`, `c mapY`, `c x`, `c y`, `c w`, `c h`, then w×h `s tile` row-major, then ignored padding: bytes after the tiles are not read, a payload one byte short of 6+2*wh* is dropped silently, a block past column 63 is clipped (2.22+; client reaction lab-tested with injected frames, no server seen sending it) **[bin][cap]**
187	0xdb	`PLO_UPDATEPACKAGEISUPDATED`	·	package-update message (5.07, 6.037)
188	0xdc	`PLO_NC_CLASSDELETE`	·	`class`; client 5.07+ ignores it
189	0xdd	`PLO_MOVE2`	·	`i3 id`; as 165 with an extra flag (2.31 only) **[bin]**

Id	Wire	Name	2.05	Notes (restated from server source)
190	0xde	`PLO_SERVERLISTCONNECTED`	·	
191	0xdf	`PLO_SHOOT2`	·	5.07 / 6.037
192	0xe0	`PLO_NC_WEAPONGET`	·	`str name`, `str image`, `script`; client 5.07+ ignores it
193	0xe1	`PLO_UNKNOWN193`	·	`i5 ?`, `?`
194	0xe2	`PLO_CLEARWEAPONS`	·	empty; precedes the weapon list
195	0xe3	`PLO_LOADGANI`	·	`str gani`, `list` (SETBACKTO and others)
197	0xe5	`PLO_LOADSCRIPT`	·	`list type,name,saveToDisk,desKey,crc32` (type weapon/npc/class/gani; key = two i5)
198	0xe6	`PLO_SERVEROPTIONS`	·	`i5 serverOptions`
252	—	`PLO_SET_ENC_KEY`	·	sets AES/RC4 key and IV
253	—	`PLO_BUNDLE`	·	several packets in one

Layouts of ids 25, 39, 41, 44, 47, 49, 68–84, 150–156, 165–189 [bin] were read from the 2.17.1, 2.22 and 2.31 receive handlers (decompile pass of 2026-10-24); the full table with version gates and confidence is `version-profiles/plo-layouts-217-231.tsv`, and the handled-id set of every build from 2.00 to 2.31 is `version-profiles/plo-handled-ids-by-build.tsv`. Dispatch bounds: 2.17.1 id 185, 2.22 id 186, 2.31 id 189. Ids 152 and 167 are not handled by any of them. Id 26 (`PLO_NPCACTION`) is handled by 2.00–2.03 and dropped in 2.04. Still undecoded: the extra step 2.31 adds after `PLO_SETACTIVELEVEL` (156); see `client-internals/decomp-findings-2026-10-24-b/05-open-items.md`.

No jump-table entry in 2.05: 26 (`PLO_NPCACTION`), 49–74, 76–99, and everything ≥ 101 except 150, 151, 153–156, 165, 166, 168. Ids 101 and 102 are read behind `PLO_RAWDATA` (see `codec-and-framing.md`). The dispatcher bound is 0xA8 (id 168). Unused slots fall to one shared default arm. **[2.05]**

Client → server (PLI_*)

Id	Wire	Name	2.05	Notes (restated from server source)
0	0x20	`PLI_LEVELWARP`	✓	
1	0x21	`PLI_BOARDMODIFY`	✓	
2	0x22	`PLI_PLAYERPROPS`	✓	
3	0x23	`PLI_NPCPROPS`	✓	
4	0x24	`PLI_BOMBADD`	✓	
5	0x25	`PLI_BOMBDEL`	✓	v6+ also sends it around login
6	0x26	`PLI_TOALL`	✓	2.05: chat text with a `toall:` prefix
7	0x27	`PLI_HORSEADD`	✓	
8	0x28	`PLI_HORSEDEL`	✓	
9	0x29	`PLI_ARROWADD`	✓	
10	0x2a	`PLI_FIRESPY`	✓	
11	0x2b	`PLI_THROWCARRIED`	✓	

Id	Wire	Name	2.05	Notes (restated from server source)
12	0x2c	`PLI_ITEMADD`	✓	
13	0x2d	`PLI_ITEMDEL`	✓	
14	0x2e	`PLI_CLAIMPKER`	✓	
15	0x2f	`PLI_BADDYPROPS`	✓	
16	0x30	`PLI_BADDYHURT`	✓	
17	0x31	`PLI_BADDYADD`	✓	
18	0x32	`PLI_FLAGSET`	✓	
19	0x33	`PLI_FLAGDEL`	✓	
20	0x34	`PLI_OPENCHEST`	✓	
21	0x35	`PLI_PUTNPC`	✓	
22	0x36	`PLI_NPCDEL`	✓	
23	0x37	`PLI_WANTFILE`	✓	
24	0x38	`PLI_SHOWIMGPLAYER`	✓	2.31 also sends it by itself about every 5.1 s with a 1-byte payload (value 9 seen); 2.22 does the same only while another player is in the level; the server relays it unchanged as 32; meaning unknown **[cap]**
25	0x39	`PLI_UNKNOWN25`	·	
26	0x3a	`PLI_HURTPLAYER`	✓	
27	0x3b	`PLI_EXPLOSION`	✓	
28	0x3c	`PLI_PRIVATEMESSAGE`	✓	
29	0x3d	`PLI_NPCWEAPONDEL`	✓	
30	0x3e	`PLI_LEVELWARPMOD`	✓	
31	0x3f	`PLI_PACKETCOUNT`	·	
32	0x40	`PLI_ITEMTAKE`	✓	
33	0x41	`PLI_WEAPONADD`	✓	
34	0x42	`PLI_UPDATEFILE`	✓	
35	0x43	`PLI_ADJACENTLEVEL`	✓	
36	0x44	`PLI_HITOBJECTS`	✓	
37	0x45	`PLI_LANGUAGE`	✓	
38	0x46	`PLI_TRIGGERACTION`	✓	`i3 fromNPC`, `c x`, `c y`, `list actionAndParams`; 2.05 sends x and y in half-tiles (see `packet-layouts.md`)
39	0x47	`PLI_TAMPERCHECK`	·	10 raw bytes, a constant per client build (not two `i5`); sent right after `PLI_LANGUAGE` at login. 2.22 also re-sends it on every gmap edge crossing, 2.31 about every 3.2 s. Values in `version-profiles/tampercheck-values.tsv`; how the bytes are built is in `client-internals/decomp-findings-2026-10-24-b/01-tamper-check-id39.md` **[cap][bin]**
40	0x48	`PLI_SHOOT`	·	

Id	Wire	Name	2.05	Notes (restated from server source)
41	0x49	`PLI_SERVERWARP`	·	server name; the gserver asks the listserver and answers with `PLO_SERVERWARP` (178) (lab, request injected) **[cap]**
43	0x4b	`PLI_MUTEPLAYER`	·	player-list mute: `s player id`, `c on/off` (1 = mute, 0 = unmute); one message per click, confirmed from a real 2.22 client **[cap]**
44	0x4c	`PLI_PROCESSLIST`	·	the client's reply to `PLO_LISTPROCESSES` (182) **[bin][cap]**
45	0x4d	`PLI_PROCESSRUNNING`	·	`c flag`: reply to HASPROCESSRUNNING; 1 if the server's string occurs in the client's own process list **[bin]**
46	0x4e	`PLI_ENTERLEVEL`	·	`c 1` on every level entry (login, gmap edge crossings, doors, edge links, server warps in the lab), sent just before the first props of the new level; 2.31 only (2.22 sends none), and 2.31 does not re-send id 39 there **[cap]**
47	0x4f	`PLI_VERIFYWANTSEND`	·	`i5 crc32`, `str filename`: send the file if stale
48	0x50	`PLI_SHOOT2`	·	
50	0x52	`PLI_RAWDATA`	·	
51	0x53	`PLI_RC_SERVEROPTIONSGET`	·	
52	0x54	`PLI_RC_SERVEROPTIONSSET`	·	
53	0x55	`PLI_RC_FOLDERCONFIGGET`	·	
54	0x56	`PLI_RC_FOLDERCONFIGSET`	·	
55	0x57	`PLI_RC_RESPAWNSET`	·	
56	0x58	`PLI_RC_HORSELIFESET`	·	
57	0x59	`PLI_RC_APINCREMENTSET`	·	
58	0x5a	`PLI_RC_BADDYRESPAWNSET`	·	
59	0x5b	`PLI_RC_PLAYERPROPSGET`	·	
60	0x5c	`PLI_RC_PLAYERPROPSSET`	·	
61	0x5d	`PLI_RC_DISCONNECTPLAYER`	·	
62	0x5e	`PLI_RC_UPDATELEVELS`	·	
63	0x5f	`PLI_RC_ADMINMESSAGE`	·	
64	0x60	`PLI_RC_PRIVADMINMESSAGE`	·	
65	0x61	`PLI_RC_LISTRCS`	·	
66	0x62	`PLI_RC_DISCONNECTRC`	·	
67	0x63	`PLI_RC_APPLYREASON`	·	
68	0x64	`PLI_RC_SERVERFLAGSGET`	·	
69	0x65	`PLI_RC_SERVERFLAGSSET`	·	
70	0x66	`PLI_RC_ACCOUNTADD`	·	
71	0x67	`PLI_RC_ACCOUNTDEL`	·	
72	0x68	`PLI_RC_ACCOUNTLISTGET`	·	
73	0x69	`PLI_RC_PLAYERPROPSGET2`	·	by player id

Id	Wire	Name	2.05	Notes (restated from server source)
74	0x6a	PLI_RC_PLAYERPROPSGET3	·	by account name
75	0x6b	PLI_RC_PLAYERPROPSRESET	·	
76	0x6c	PLI_RC_PLAYERPROPSSET2	·	
77	0x6d	PLI_RC_ACCOUNTGET	·	
78	0x6e	PLI_RC_ACCOUNTSET	·	
79	0x6f	PLI_RC_CHAT	·	
80	0x70	PLI_PROFILEGET	✓	
81	0x71	PLI_PROFILESET	✓	
82	0x72	PLI_RC_WARPPLAYER	·	
83	0x73	PLI_RC_PLAYERRIGHTSGET	·	
84	0x74	PLI_RC_PLAYERRIGHTSSET	·	
85	0x75	PLI_RC_PLAYERCOMMENTSGET	·	
86	0x76	PLI_RC_PLAYERCOMMENTSSET	·	
87	0x77	PLI_RC_PLAYERBANGET	·	
88	0x78	PLI_RC_PLAYERBANSET	·	
89	0x79	PLI_RC_FILEBROWSER_START	·	
90	0x7a	PLI_RC_FILEBROWSER_CD	·	
91	0x7b	PLI_RC_FILEBROWSER_END	·	
92	0x7c	PLI_RC_FILEBROWSER_DOWN	·	
93	0x7d	PLI_RC_FILEBROWSER_UP	·	
94	0x7e	PLI_NPCSERVERQUERY	·	
96	0x80	PLI_RC_FILEBROWSER_MOVE	·	
97	0x81	PLI_RC_FILEBROWSER_DELETE	·	
98	0x82	PLI_RC_FILEBROWSER_RENAME	·	
103	0x87	PLI_NC_NPCGET	·	`i3 id`
104	0x88	PLI_NC_NPCDELETE	·	`i3 id`
105	0x89	PLI_NC_NPCRESET	·	`i3 id`
106	0x8a	PLI_NC_NPCSCRIPTGET	·	`i3 id`
107	0x8b	PLI_NC_NPCWARP	·	`i3 id`, `c x*2`, `c y*2`, `level`
108	0x8c	PLI_NC_NPCFLAGSGET	·	`i3 id`
109	0x8d	PLI_NC_NPCSCRIPTSET	·	`i3 id`, `str script`
110	0x8e	PLI_NC_NPCFLAGSSET	·	`i3 id`, `str flags`
111	0x8f	PLI_NC_NPCADD	·	`str info` = name,id,type,scripter,level,x,y
112	0x90	PLI_NC_CLASSEDIT	·	`class`
113	0x91	PLI_NC_CLASSADD	·	`c name length`, `name`, `str script`
114	0x92	PLI_NC_LOCALNPCSGET	·	`level`
115	0x93	PLI_NC_WEAPONLISTGET	·	
116	0x94	PLI_NC_WEAPONGET	·	`weapon`
117	0x95	PLI_NC_WEAPONADD	·	`c weapon length`, `weapon`, `c image length`, `image`, `code`
118	0x96	PLI_NC_WEAPONDELETE	·	`weapon`

Id	Wire	Name	2.05	Notes (restated from server source)
119	0x97	PLI_NC_CLASSDELETE	·	class
130	0xa2	PLI_REQUESTUPDATEBOARD	·	c level length, level, i5 modtime, s x, s y, s width, s height
150	0xb6	PLI_NC_LEVELLISTGET	·	
151	0xb7	PLI_NC_LEVELLISTSET	·	str levels
152	0xb8	PLI_REQUESTTEXT	·	read a server value
154	0xba	PLI_SENDTEXT	·	write a server value
155	0xbb	PLI_RC_LARGEFILESTART	·	
156	0xbc	PLI_RC_LARGEFILEEND	·	
157	0xbd	PLI_UPDATEGANI	·	gani update request (like UPDATECLASS)
158	0xbe	PLI_UPDATESCRIPT	·	rest = script name to fetch
159	0xbf	PLI_UPDATEPACKAGEREQUESTFILE	·	order?, package file name, modtime
160	0xc0	PLI_RC_FOLDERDELETE	·	
161	0xc1	PLI_UPDATECLASS	·	i5 modtime + rest = class name
162	0xc2	PLI_RC_UNKNOWN162	·	empty; RC3 beta
252	—	PLI_SET_ENC_KEY	·	sets AES/RC4 key and IV
253	—	PLI_BUNDLE	·	several packets in one

2.05 senders were found for 0–5, 7–24, 26–30, 32–38, 80, and 81 **[2.05]**. A sender for 6 (PLI_TOALL) was also found: chat-bar text starting with toall: is sent as id 6 (see chat-pm-rc.md). No sender was found for 25 or 31 (PLI_PACKETCOUNT).

Id 0 is overloaded. The login line starts with GCHAR 0 because that byte is the client type, and later PLI_LEVELWARP messages also use id 0. The server tells them apart by connection state. **[srv][2.05]**

Renames between gs2lib versions

The GServer-v2 submodule copy (2025-01) uses older names for several ids: PLI_SHOWIMG → PLI_SHOWIMGPLAYER (24), PLI_MAPINFO → PLI_TAMPERCHECK (39), PLI_UNKNOWN46 → PLI_ENTERLEVEL (46), PLO_SHOWIMG → PLO_SHOWIMGPLAYER (32), PLO_FULLSTOP/2 → PLO_KINGDOMSMODE/PLO_NEWWORLDMODE (176/177), PLO_UNKNOWN183–186 → HASPROCESSRUNNING/TAKESCREENSHOT/BOARDHEIGHTS/BOARDMODIFY2, PLO_UNKNOWN190 → PLO_SERVERLISTCONNECTED. The newer copy also adds PLI_PROCESSRUNNING 45, PLI_SET_ENC_KEY 252, PLI_BUNDLE 253, PLO_SETNETCOOKIE 111, and PLO_UPGRADEURL 169.

Conflicts with older notes

- OPCODE-SUMMARY.md (39ster 2.1 source) names 100/101/102 FILELEN/TILES/FILEBLOB. gs2lib calls them RAWDATA/BOARDPACKET/FILE. The roles are the same. Its other server→client ids agree with gs2lib.
- session-opcodes-cleaned.md lists 0x22 as "Computer / NPC-like object". 0x22 is PLO_BADDYPROPS (baddies are "compus" in GS1). NPCs are 0x23.
- The same table's 0x85 "level load (raw path)" and 0x86 "stream data" are 101 PLO_BOARDPACKET and 102 PLO_FILE, both read behind PLO_RAWDATA (0x84).

- `packet-id-diff.md` provisional ids (`Login 0x01`, `LevelRequest 0x10`, `PlayerUpdate 0x20`) are retired. The real ids are login type 0, `PLI_LEVELWARP` 0, and `PLI_PLAYERPROPS` 2.

2.05 receive count [2.05]: 59 handled PLO ids plus a silent default arm. Every id marked · above is ignored without error, including 26 and 49–59. UDP peer messages use a separate id space; see `udp-p2p.md`.

`protocol/packet-layouts.md`

Packet layouts

Date: 2026-10-14, revised 2026-10-21 (2.05 trace) and 2026-10-24 (ids above 48)

Sources: GServer-v2 `TLevel*.cpp`, `TPlayer.cpp`, `TWeapon.cpp` (GPLv3, summarized); client 2.05 rev 2 PLO dispatcher and PLI senders. Tags: **[srv]** server source · **[2.05]** read from the 2.05 receiver or sender · **[inf]**.

Notation: `c` GCHAR, `s` GSHORT, `i3` GINT3, `i4` GINT4, `i5` GINT5, `str` GSTRING (`c len` + bytes), `rest` = the rest of the message. `x×2` means the value is in half-tiles (the client divides by 2). Ids are logical, so add 0x20 for the wire byte. Ids and names are in `packet-ids.md`, prop encodings in `player-props.md`, UDP ids in `udp-p2p.md`.

How 2.05 reads a message [2.05]

- Every handler checks a minimum length first and drops a short message silently. The minimums are given below where they matter.
- 2.05 keeps two level pointers. The **named level** is set by every LEVELNAME. The **object level** receives NPC packets (NPCPROPS, NPCDEL2, HIDENPCS, SHOWIMGNPC); SETACTIVELEVEL sets it, and without an NPC server each LEVELNAME sets it too.
- Handlers for level objects (bombs, arrows, items, explosions, hits, trigger actions, horses, push-away) only act when the player is standing in the named level.
- Packets that the server re-emits for another player start with `s player id`. With an unknown id, bombs, arrows, explosions and items are still shown (without an owner), while fire spy, throw carried and player hits are dropped.

Level data (server → client)

Id	Name	Layout	Tag
6	LEVELNAME	rest = level filename. Sets the named level, and also the object level when there is no NPC server (no HASNPCSERVER)	[srv][2.05]
100 → 101	RAWDATA → BOARDPACKET	`i3(8194)`, then `c(101)` + 4096 × u16 **little-endian raw** tile ids + `\n`	[srv]
0	LEVELBOARD	a run of board changes, each `c x, c y, c w, c h` + w × h × `s tile` (row-major), back to back with no separator. Parsing stops when fewer than 4 bytes are left or a run is shorter than `w*h*2+4` (the rest is dropped). Empty = "use your cached board". If a warp to this level is pending, 2.05 completes the warp first and then applies the runs	[srv][2.05]
7	BOARDMODIFY	one run: `c x, c y, c w, c h` + w × h × `s tile`, row-major; needs `len ≥ w*h*2+4`	[srv][2.05]

Id	Name	Layout	Tag
1	LEVELLINK	rest = `"dest x y w h newx newy"`, split on spaces (`newx`/`newy` may be `playerx`/`playery`)	[srv][2.05]
5	LEVELSIGN	`c x, c y` (whole tiles) + sign-alphabet text (see `formats/sign-text.md`); needs ≥ 2 bytes	[srv][2.05]
4	LEVELCHEST	`c opened`, `c x`, `c y` (whole tiles); if not opened, also `c item`, `c signindex` (needs ≥ 5 bytes, else dropped)	[srv][2.05]
2	BADDYPROPS	`c id` + BDPROP stream. An unknown id creates the baddy	[srv][2.05]
3	NPCPROPS	`i3 id` + NPCPROP stream; needs id > 0. Ids below 1000 are level NPCs (created in the object level), 1000 and up are server-wide NPCs. A new NPC gets the `playerenters` event if the player is in that level	[srv][2.05]
17	HORSEADD	`c x×2, c y×2, c (dir & 3) + 4 × bushes` (bushes capped at 6) + rest = image (`.gif` is added if there is no extension)	[srv][2.05]
18	HORSEDEL	`c x×2, c y×2`; removes any horse within 2 tiles of that point	[2.05]
22	ITEMADD	`c x×2, c y×2, c item` (0–24, unknown items ignored). If the message is longer than 4 bytes, bytes 3–4 are `s player`: a dropped sword or shield then copies that player's image	[srv][2.05]
23	ITEMDEL	`c x×2, c y×2`; removes the item at exactly that point	[srv][2.05]
39	LEVELMODTIME	`i5 modtime`, exactly 5 bytes. 2.05 then writes the level to its web-level cache (`GWEBL001` format, see `formats/gwebl001-cache.md`)	[srv][2.05]
10	ISLEADER	empty: you run this level's baddies and timeouts. Ignored unless the player's level is the active level	[srv][2.05]
42	NEWWORLDTIME	`i4 time` (`b0<<21 | b1<<14 | b2<<7 | b3`); needs ≥ 4 bytes	[srv][2.05]
156	SETACTIVELEVEL	rest = level that receives the following NPC packets (the object level)	[srv][2.05]

Pre-2.1 level sequence (`sendLevel141`, used for 2.05) [srv]

1. If the client's cached modtime matches: board changes since then, and nothing else.
2. Otherwise: `RAWDATA` + `BOARDPACKET`, then `LEVELNAME` (first level only), `LEVELLINK`×n, `LEVELSIGN`×n, `LEVELMODTIME`.
3. Then board changes, `LEVELCHEST`×n, `HORSEADD`×n, `BADDYPROPS`×n, `ISLEADER` (if leader), `NEWWORLDTIME`, `NPCPROPS`×n, and `OTHERPLPROPS` for each player in the level.

Board modify correction

Earlier notes had `x:u8 y:u8 width:u8 height:u8 tiles:raw`. The header bytes are **GCHAR**s (+0x20), and each tile is a **GSHORT** (two 7-bit bytes, +0x20). Only `BOARDPACKET` (101) carries raw little-endian u16 tiles. **[srv]** The same shape is used client → server in `PLI_BOARDMODIFY` (1) for bush or sign pickups; 2.05 sends exactly one run. **[2.05]** The server rebroadcasts it as `PLO_BOARDMODIFY`.

Board change on the wire [cap]

Lab capture, two 2.05 clients in one level (`tools/lab-setup.md`). Player B dropped a bomb next to a patch of bushes and grass.

1. **Bomb:**
   - B sent `PLI_BOMBADD` (4): `c x×2, c y×2, c owner_power, c fuse`. The values seen were x 78, y 28, `owner_power` 9 and fuse 55.
   - The server sends `PLO_BOMBADD` (11) to the other players: `s player id` followed by the same four bytes.
   - The 2.05 client writes the third byte as `power + 8`, where the 8 flag means "this bomb is mine". A receiver reads the power as `v & 7` and the owner flag from the value 8 (bit 3). So 9 is power 1 plus the owner flag. GServer's own reading, `(player << 2) | power`, does not match 2.05. **[2.05]** The fuse 55 is probably 0.05 s steps, about 2.75 s. **[inf]**
2. **Board change:**
   - When the bomb went off, **player A** sent eight `PLI_BOARDMODIFY` (1) messages in one frame, one per destroyed 2×2 patch. B, who owned the bomb, sent none. These sessions had no `ISLEADER`, so why A was the one to report is not settled. **[inf]**
   - Each message is `c x, c y, c w, c h` in whole tiles, then w × h GSHORT tile ids in row order. For example, x 39, y 14, 2 × 2, tiles 1834 1835 1850 1851 (a cut bush). A cut grass patch gave 677 678 693 694.
3. **Rebroadcast:**
   - In the same tick the server sent `PLO_BOARDMODIFY` (7) with exactly the same bytes to **both** clients, including the sender.
   - It interleaved them with its own `PLO_ITEMADD` (22) drops: `c x×2, c y×2, c item`, with items 1 and 4 seen. The drops come from the server, because `bushitems` was on. No client sent `PLI_ITEMADD`.
4. **Pickup:**
   - B walked over the drops and sent `PLI_ITEMDEL` (13): `c x×2, c y×2` for each.
   - The server sent `PLO_ITEMDEL` (23) to A only, not back to B.
5. **Respawn:**
   - Exactly 15.0 s after the change (`respawntime = 15`), the server sent `PLO_BOARDMODIFY` again for every patch, with the original tiles: bush 34 35 50 51, grass 2 3 18 19.
   - Clients do not request the respawn. It is pushed by the server. This is GServer-v2's fixed timer. The original GServer13 used a random 6–12 s for the default `respawntime` 10, only for 2 × 2 changes, and did not echo a change to its sender: see `client-internals/board-respawn.md`.
6. **Level entry:** an empty `PLO_LEVELBOARD` (0) followed the board packet and `LEVELMODTIME` on level entry, as described above.

Player state (server → client)

Id	Name	Layout	Tag
9	PLAYERPROPS	PLPROP stream (self)	[srv][2.05]
8	OTHERPLPROPS	`s id` + PLPROP stream; needs ≥ 2 bytes. An unknown id creates the player	[srv][2.05]
14	PLAYERWARP	`c x×2, c y×2` + rest = level. The warp stays pending until the level data arrives	[srv][2.05]
15	WARPFAILED	rest = level; 2.05 ignores the text and just shows a short failed-warp state	[srv][2.05]

Id	Name	Layout	Tag
16	DISCMESSAGE	rest = text, stored as the disconnect reason; the server then closes the connection	[srv][2.05]
40	HURTPLAYER	`s attacker` + `c dx, c dy, c power` [+ `i3 npc`]: see "Hits" below	[srv][2.05]
38	PUSHAWAY	`c dx, c dy`, each `v/16 − 4`; ignored unless the player's level is the active level	[srv][2.05]
28 / 31	FLAGSET / FLAGDEL	rest = `name` or `name=value`. FLAGSET adds a flag only if it is not already set	[srv][2.05]
154 / 155	FREEZEPLAYER2 / UNFREEZEPLAYER	empty. Freeze lasts 600 ticks unless unfrozen	[srv][2.05]
153	SAY2	rest = text, `\xa7` = newline	[srv][2.05]

Combat and objects (both directions)

The client sends its own action. The server re-emits it to the rest of the level with `s player id` added in front. **[srv]** In 2.05 most of these also go peer to peer over UDP; the TCP copy is sent only when some player in the level has no UDP port, or when the server has an NPC server (HASNPCSERVER). See `udp-p2p.md`. **[2.05]** This rule covers these level-object packets only. Player props still go over TCP while UDP is active (seen in the lab captures, `tools/lab-setup.md`).

PLI	PLO	Name	Client layout (PLI)	Server copy as 2.05 reads it (PLO)	Tag
4	11	BOMB ADD	`c x×2, c y×2, c power (+8 = the bomb is the sender's), c fuse`	`s player` + the same 4 bytes; power = `v & 7`, value 8 (bit 3) = owned by that player	[srv] [2.05]
5	12	BOMB DEL	`c x×2, c y×2`	**no player id**: `c x×2, c y×2`; removes the bomb at exactly that point	[srv] [2.05]
9	19	ARROWADD	`c x×2, c y×2, c flags, c sprite`, then the projectile type (0 arrow, 1 fireball, 2 fireblast, 3 nuke) sent as `v + 33` instead of `v + 32`; flags = `dir & 3`, +4 first removes a matching shot, +8 owned	`s player` + the same 5 bytes; needs ≥ 5. Receiver rules: `client-internals/damage-sources.md`, "Other players' bombs, blasts and shots"	[srv] [2.05]
10	20	FIRESPY	`c (a × 8 + b)`	`s player` + that byte; the receiver uses only `a`	[srv] [2.05]
11	21	THROWCARRIED	empty	`s player` only	[srv] [2.05]
26	40	HURTPLAYER	`s victim, c (dx+4)×16, c (dy+4)×16, c power` [+ `i3 npc`]	`s attacker` + `c dx, c dy, c power` [+ `i3 npc`]	[srv] [2.05]
27	36	EXPLOSION	`c radius, c x×2, c y×2, c power (+8 = the explosion is the sender's)`	`s player` + the same 4 bytes; power `& 7` clamped to 1–3, value 8 (bit 3) = owned	[srv] [2.05]
36	46	HITOBJECTS	`c power×2, c x×2, c y×2` [+ `i3 npc`]	`s player` + the same bytes	[srv] [2.05]
15	2	BADDYPROPS	rest = `c id` + BDPROP stream (passed through)	`c id` + BDPROP stream	[srv] [2.05]
16	27	BADDYHURT	`c id, c (dx+4)×16, c (dy+4)×16, c power`	same; applied only by the level leader	[srv] [2.05]
17	—	BADDYADD	`c x×2, c y×2, c type, c power` + rest = image	(the server answers with BADDYPROPS)	[srv] [2.05]
20	4	OPENCHEST	`c x, c y` (whole tiles); server replies LEVELCHEST opened=1	see LEVELCHEST	[srv] [2.05]

PLI	PLO	Name	Client layout (PLI)	Server copy as 2.05 reads it (PLO)	Tag
12 / 13	22 / 23	ITEMADD / ITEMDEL	ITEMADD `c x×2, c y×2, c item`; ITEMDEL `c x×2, c y×2`	see the level table	[srv] [2.05]
32	—	ITEMTAKE	`c x×2, c y×2, c item`	—	[2.05]
38	48	TRIGGERACTION	`i3 npc` (0 = none), `c x×2, c y×2` + rest = `action` or `action,params`	`s player` + the same; needs > 5 bytes	[srv] [2.05]

Hits [2.05]

- **HURTPLAYER (40)**: `dx`, `dy` decode as `(v − 0x60) / 16` (knockback, −4..4) and `power` as `(v − 32) / 2` hearts, capped at 1.0. With 6 or more bytes after the player id, the hit came from an NPC (`i3`), and distance is measured from that NPC. The hit is applied only if the attacker (player or NPC) is within 10 tiles and the local player can be hurt at that moment.
- **HITOBJECTS (46)**: `power` is `(v − 32) / 2`, capped at 1.0, then `x`, `y` in half-tiles, and an optional `i3 npc`. It hurts the local player only when the player's centre is within 2 tiles of `(x, y)` in each axis.
- Both share one handler and one flag that picks the layout. The same handler also takes UDP id 21 (GS1 `hitplayer`).
- **TRIGGERACTION (48)** fires the `action` event on the local player's scripts only when the source (the NPC if `npc > 0`, else the sending player) is within 10 tiles. The split is at the first comma.

Level navigation (client → server)

Id	Name	Layout	Tag
0	LEVELWARP	`c x×2, c y×2` + rest = level	[srv][2.05]
30	LEVELWARPMOD	`i5 modtime, c x×2, c y×2` + rest = level. 2.05 sends this instead of LEVELWARP when it has the level cached (in memory or in its web-level cache), so the server can answer with only the changes	[srv][2.05]
35	ADJACENTLEVEL	`i5 modtime` + rest = level (gmap preload). The modtime is all zeros when the level is not cached	[srv][2.05]

In most states, a warp to the level the player is already in just moves the player and sends nothing. A real warp also sends UDP id 24 to gmap neighbours. **[2.05]**

Other client → server senders [2.05]

Id	Name	Layout
2 / 3	PLAYERPROPS / NPCPROPS	rest = prop stream (passed through)
7 / 8	HORSEADD / HORSEDEL	HORSEADD `c x×2, c y×2, c dir + 4 × bushes` + image filename; HORSEDEL `c x×2, c y×2`
14	CLAIMPKER	`s player id`
18 / 19	FLAGSET / FLAGDEL	rest = flag; flags that start with `local.` are never sent
21	PUTNPC	`str image`, `str script file`, `c x×2, c y×2`
22	NPCDEL	`i3 npc id` (only for NPCs with id > 0). If the NPC is one of the player's weapons, 2.05 sends NPCWEAPONDEL (29) with the weapon name instead
23	WANTFILE	rest = filename

Id	Name	Layout
24	SHOWIMG	rest (passed through unchanged as PLO 32 `GSHORT sender id` + rest; also UDP 15). 2.31 sends a 1-byte value 9 about every 5.1 s; 2.22 only with another player in the level
34	UPDATEFILE	`i5 modtime` + filename
80 / 81	PROFILEGET / PROFILESET	see `chat-pm-rc.md`

Weapons

Id	Name	Layout	Tag
PLO 33	NPCWEAPONADD	`str name` + NPCPROP stream (normally `c 0 (IMAGE)` + `str image`, `c 1 (SCRIPT)` + `s len` + GS1 text). A name the player already has is ignored. The weapon gets the `playerenters` event	[srv][2.05]
PLO 34	NPCWEAPONDEL	rest = name	[srv][2.05]
PLO 43	DEFAULTWEAPON	`c item index` (0–24; built-in weapon: bow, bomb, sword, …)	[srv][2.05]
PLI 33	WEAPONADD	`c type`: 0 → `c item index`; 1 → `i3 npc id` (copy an NPC's script as a weapon). **2.05 only sends type 0**	[srv][2.05]
PLI 29	NPCWEAPONDEL	rest = name	[srv][2.05]

For v4+ clients the script prop is replaced by `NPCPROP_CLASS` + `PLO_UNKNOWN197` (GS2 bytecode). 2.05 never sees that path. **[srv]**

GANI

The GANI file format is in `formats/gani.txt`. On the wire, animation is only a prop: `PLPROP_GANI` (10) and `NPCPROP_GANI` (12) carry the GANI name, and the client fetches the file with `PLI_WANTFILE` like any other asset. Pre-2.1 clients (2.05) use slot 10 as BOWGIF. **[srv]** Gani attribute props (`PLPROP_GATTRIB*`) are strings that GANI `PARAM`/`ATTR` placeholders resolve against. **[inf]** 2.05 sends and receives GATTRIB1–5 (props 37–41, `#P1`–`#P5`), but its expression table has no `gattrib` entry. **[2.05]**

Differences from GServer worth knowing [2.05]

- PLO BOMBDEL, HORSEDEL and ITEMDEL carry no player id. Only the coordinates are read.
- PLO HURTPLAYER and HITOBJECTS take an optional trailing `i3 npc`.
- The arrow's last byte is offset by 33 (decoded as `b − 33`), not a plain GCHAR.
- TRIGGERACTION coordinates are half-tiles, not whole tiles.

Server → client ids above 48 and related (2.17.1, 2.22, 2.31) [bin]

Read from the receive handlers of the three clients (decompile pass of 2026-10-24); behaviour notes, version gates and confidence per id are in `version-profiles/plo-layouts-217-231.tsv`, and the narrative is in `client-internals/decomp-findings-2026-10-24-b/02-server-to-client-layouts.md`. Number encodings are as above: GCHAR = byte − 0x20, GSHORT two 7-bit bytes, GINT3/4/5 three to five 7-bit bytes; "comma text" goes through the client's comma-text parser (double quotes allowed). Ids not listed fall into the shared default arm and are ignored. Id 39 is shown as the 2.17+ handlers read it (the rest is handed unparsed to the active level object); the 2.05 handler reads an `i5 modtime` and caches the level (table above). The handlers check a minimum length and drop short messages silently.

Id	Name	Present in	Payload
25	SIGNATURE	all three	1 GCHAR v
39	LEVELMODTIME	all three	raw rest, not parsed here
41	STARTMESSAGE	all three	raw text (RTF or HTML)
44	HASNPCSERVER	2.17,2.22 only	none
46	(PLO 46)	all three	GSHORT id, rest
47	STAFFGUILDS	all three	comma text
49	PLAYERWARP2	2.17 2.22 2.31 (also 2.1 test builds)	GCHAR x2, GCHAR y2, GCHAR z(ignored here), GCHAR gmapX, GCHAR gmapY, rest = level name
68	LARGEFILESTART	all three	raw name
69	LARGEFILEEND	all three	raw name
75	PROFILE	all three	repeat: GCHAR n, n chars (trimmed)
84	LARGEFILESIZE	2.31 only	GINT5
100	RAWDATA	all three	GINT3 length
150	NPCDEL2	all three	GCHAR n, level name (n chars), GINT3 npc id
151	HIDENPCS	all three	none
153	SAY2	all three	raw text
154	FREEZEPLAYER2	all three	none
155	UNFREEZEPLAYER	all three	none
156	SETACTIVELEVEL	all three	raw level or .gmap name
165	MOVE	all three	GINT3 npc id, rest = move data
166	SHOWIMGNPC	all three	GINT3 npc id, rest = props
168	UNKNOWN168	all three	none read
169	UPGRADEURL	all three	raw text
170	GHOSTMODE	all three	GCHAR flag
171	BIGMAP	all three	comma text >=2 fields (>=4 to be used): txt, png, x, y
172	MINIMAP	all three	same as 171
173	GHOSTTEXT	all three	raw text
174	GHOSTICON	all three	GCHAR flag
175	SHOOT	all three	GSHORT player id, rest = shot data
176	KINGDOMSMODE	all three	none
177	NEWWORLDMODE	all three	none
178	SERVERWARP	all three	comma text, at least 4 fields: field 0 unused, 1 = name (not needed to connect), 2 = host, 3 = port; fewer than 4 fields is ignored (2.31 lab)
179	RPGWINDOW	all three	raw text
180	STATUSLIST	all three	raw text
181	UNKNOWN181	all three	GSHORT player id, GCHAR toggle
182	LISTPROCESSES	all three	none
183	HASPROCESSRUNNING	all three	raw list of names
184	TAKESCREENSHOT	all three	raw file name
185	BOARDHEIGHTS	all three	GCHAR mapX, GCHAR mapY, GCHAR blockX, GCHAR blockY, GCHAR w-1, GCHAR h-1, then (w)*(h) cells of 2 raw bytes

Id	Name	Present in	Payload
186	BOARDMODIFY2	2.22,2.31 (not 2.17)	GCHAR mapX, GCHAR mapY, GCHAR x, GCHAR y, GCHAR w, GCHAR h, then w*h GSHORT tiles (row-major), rest = ignored padding (lab: not applied, no disconnect; short by one byte is dropped silently; clipped at column 63)
189	MOVE2	2.31 only	GINT3 npc id, rest